

Vision-Based Navigation Pipeline for AMRs in a Dynamic Warehouse

Minor Scientific Progress Report

WiL Project

Reporting cutoff: 16 September 2026

Contents

3	Methodology	3
3.1	Progress Scope and Implemented System Overview	3
3.2	Hardware, Software, Sensors, Calibration, and Frames	5
3.2.1	Computing and software environment	5
3.2.2	Simulated sensor rig	5
3.2.3	Simulation fidelity and its limits	7
3.2.4	Real sensor rig and calibration limits	9
3.2.5	Coordinate-frame contract	10
3.3	Environments, Datasets, and Controlled Factors	11
3.3.1	Simulation environment	11
3.3.2	Dataset registry	14
3.4	ORB-SLAM3 and Semantic Feature Filtering	16
3.5	VINS-Fusion and the RTAB-Map Back End	16
3.5.1	VINS-Fusion front end	16
3.5.2	RTAB-Map mapping and localization back end	17
3.6	Non-Visual Baseline Estimators	17
3.6.1	Implementation form and why it is offline	18
3.6.2	Sensor model derived from data	18
3.6.3	The orientation field must not be used	19
3.6.4	Estimator definitions	19
3.7	YOLO Model, Mask Construction, and Synchronization	20
3.8	Collection Protocol, Validity, and Artifacts	20
3.9	Evaluation Metrics and Analysis	22
3.9.1	Trajectory evaluation	22
3.9.2	Detection evaluation	23
3.9.3	Map and localization evaluation	23
3.9.4	Computational performance	23
3.10	Reproducibility Controls and Implementation Deviations	24
	Repository Evidence Used for This Draft	24
4	Experiments and Research Results	27
4.1	Relogged Campaign and Validity Audit	27
4.2	Non-Visual Baseline and Estimator Comparison	28
4.2.1	Question, hypotheses, and conditions	28
4.2.2	Validity	29
4.2.3	Accuracy results	31
4.2.4	Falsification of H1, and the drift mechanism	32
4.2.5	Covariance consistency	34

4.2.6	Cross-validation of the EKF noise model	35
4.2.7	Interpretation and limitations	37
4.3	Offline YOLO Detector Performance	38
4.3.1	Bounding-Box Detection and Localization	38
4.3.2	Object-Class Assignment Conditional on Localization	41
4.4	Observed Anomaly Catalogue	42
4.5	Trajectory Accuracy, Plausibility, and Robustness	42
4.5.1	Relative pose error	44
4.6	Real-Time and Computational Measurements	48
4.7	Dense Map Reconstruction and Map-Quality Evaluation	50
4.7.1	Question, scope, and conditions	50
4.7.2	Map placement as a validity precondition	50
4.7.3	Graph and database summary	51
4.7.4	Geometry against the world file	51
4.7.5	Drivability against the true route	52
4.7.6	Interpretation and limitations	55
4.7.7	Planned extension to the large warehouse	55
4.8	Loop Closure, Map Merging, and Global-Correction Evidence	56
4.9	Interpretation, Limitations, and Advisor Decisions	56
4.9.1	Where each pipeline is weak, block by block	56
	Analysis Artifacts	67

Chapter 3

Methodology

Document purpose: Minor-report draft for advisor progress review.

Reporting cutoff: 16 September 2026.

Evidence basis: Current repository source, configuration, launch files, generated artifacts, and dated engineering records.

Status: Draft. Implementation claims have been checked against the repository; literature citations and faculty-specific thesis formatting remain to be completed.

3.1 Progress Scope and Implemented System Overview

This project investigates vision-based localization for an autonomous mobile robot in a dynamic warehouse. At the reporting cutoff, the implemented work compares two stereo-inertial front ends, ORB-SLAM3 and VINS-Fusion, using common simulated or recorded sensor streams. A custom YOLO detector can be enabled to prevent image features associated with selected warehouse-object classes from entering pose estimation. RTAB-Map is used as a separate back end when persistent map construction, loop closure, pose-graph correction, or localization against a saved database is studied.

As shown in [figure 3.1](#), the current work contains two related but separate evaluation paths. The first measures front-end trajectory behaviour with and without semantic feature filtering. The second studies map reconstruction and external global correction through RTAB-Map. RTAB-Map’s `map → odom` transform does not modify the internal feature map, state history, or previously recorded trajectory of ORB-SLAM3 or VINS-Fusion. A unified architecture for relocalization, global optimization, and multi-session map merging is **planned**, not implemented. Path planning is outside the present research scope.

Table 3.1: Implementation status at the reporting cutoff.

Component	Status	Scope of evidence
Warehouse simulation and controlled worlds	Implemented; verified	Static, dynamic, textured-floor, and featureless-floor world and launch files are present.
ORB-SLAM3 ROS 2 stereo-inertial wrapper	Implemented; verified in simulation	Live topics, worker-thread tracking, TF, and common trajectory output are implemented.

Component	Status	Scope of evidence
ORB-SLAM3 semantic filtering	Implemented; mechanism verified	Detector messages reach the tracker and masked ORB candidates are rejected before octree distribution.
VINS-Fusion stereo-inertial estimator	Implemented	Simulation and real-rig configurations, odometry publication, TF, and trajectory output are present.
VINS-Fusion semantic filtering	Implemented; mechanism verified	Existing KLT tracks are culled and new features are suppressed in detected regions.
Offline YOLO evaluation	Implemented; partially evaluated	Ground-truth projection, IoU matching, per-frame CSV files, and validation overlays are present.
RTAB-Map construction and saved-map localization	Implemented; partially verified	Construction and basic read-only localization passed; one tiny-map database has been quantitatively evaluated for geometry and drivability (section 4.7). Arbitrary-start aliasing rejection and the large-warehouse evaluation have not been done.
RTAB-Map extend and multi-session merging	Implemented at launch level; experimentally unverified	The mode exists, but the planned Level 4 experiment has not been completed.
Live VINS-Fusion and RTAB-Map co-execution	Blocked	VINS diverges under the combined workload on the current computer; the mechanism is unresolved.
RPE and complete compute evaluation	Partially implemented	Front-end, back-end, local-mapping, loop-closing, delivery/state, cumulative process CPU, and resident-memory records are now logged. RPE was added after the cutoff and applied to the campaign (section 4.5.1). External-process monitoring and the repeated hardware matrix remain planned.

The repository state used for this draft is superproject commit `4425a415f624d6a45da1da4f3b2404cbf7914197`. The checked-out submodules are VINS-Fusion `4a2223c9e54dfc5d2e94eb2ec1dfc8fd3b8045bb`, RTAB-Map ROS `56064e4388f6f3ad6b994dbff98bcc0a41488102`, and the warehouse environment `3df54c6e6ec267fad204e208db61c4ab491be410`. The ORB-SLAM3 core is pinned to `5c2b00f732775f78836751dba13db25463a0aaa1`. The VINS-Fusion worktree contains an uncommitted output-directory creation fix at the

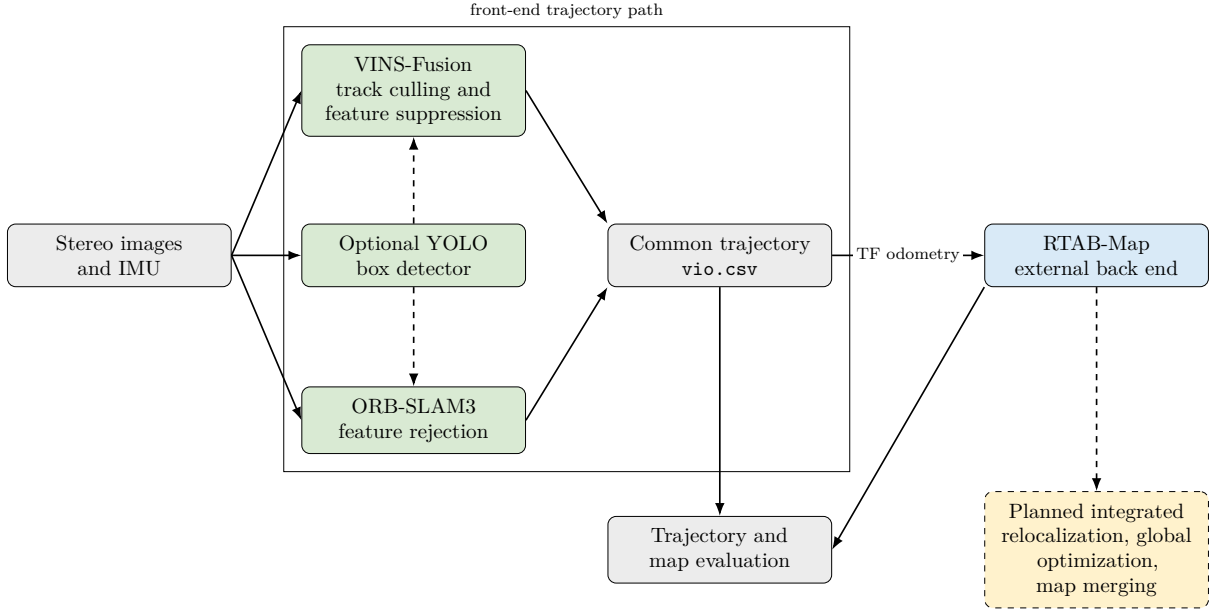


Figure 3.1: Implemented experimental architecture. Green blocks are implemented project components, blue denotes the external RTAB-Map back end, and the dashed yellow block is future work. The diagram deliberately does not show a feedback connection from RTAB-Map into either front end because that integration has not been implemented.

cutoff; this must be committed or otherwise recorded before a reproducibility release.

3.2 Hardware, Software, Sensors, Calibration, and Frames

3.2.1 Computing and software environment

Development and recorded experiments use Ubuntu 22.04.5, ROS 2 Humble, GCC 11.4.0, CMake 3.22.1, and system OpenCV 4.5.4. The recorded development machine has 16 logical CPU cores, 15 GB RAM, no swap, and an NVIDIA RTX 3070 Laptop GPU. The detector environment recorded in the engineering log contains PyTorch 2.5.1+cu121, torchvision 0.20.1, Ultralytics 8.4.138, NumPy 1.26.4, and Python OpenCV 4.11. ORB-SLAM3 is built out of tree with Pangolin 0.9.1. RTAB-Map uses the apt core library `ros-humble-rtabmap` 0.23.7 with the project checkout of its ROS wrappers; only wrapper packages through `rtabmap_slam` are built.

These specifications describe the current laptop, not a completed cross-hardware experiment. The proposed Jetson-versus-laptop study has not been instrumented or executed, so no hardware comparison is inferred from the present data.

3.2.2 Simulated sensor rig

The simulated Ackermann robot carries two ideal pinhole cameras and an IMU on a forward mast, and is driven through a front-steering, rear-driven four-wheel chassis. Figure 3.2 gives its dimensioned plan and elevation, and tables 3.2 and 3.4 list the sensor and platform constants. Every value in the figure and both tables is read from

aws-robomaker-small-warehouse-world/models/ackermann_robot/model.sdf, so the drawing is to scale rather than illustrative.

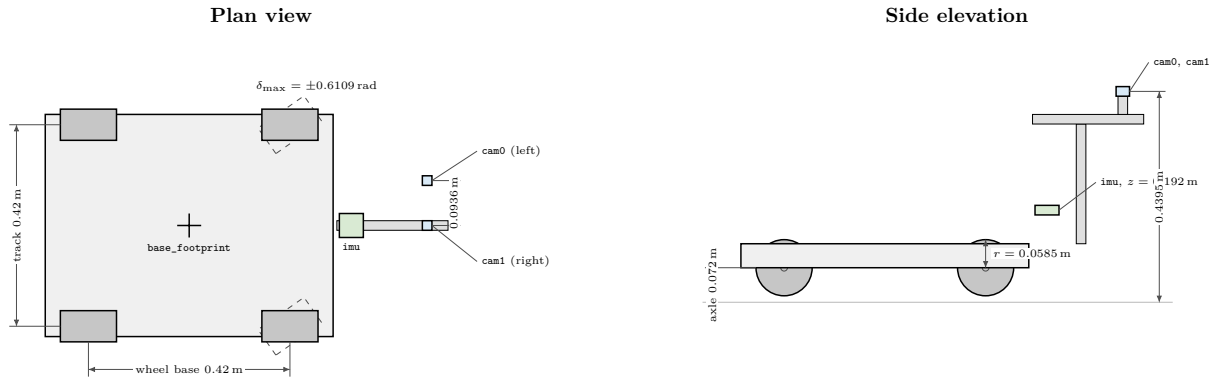


Figure 3.2: Simulated Ackermann platform, drawn to scale from the model definition. Distances are in metres relative to **base_footprint**. The stereo pair is mounted asymmetrically: **cam1** sits on the vehicle centreline and **cam0** is offset 0.0936 m to its left, so the optical centre of the pair is not the chassis centreline. The wheel base and track are equal at 0.42 m, which is what makes the rear-wheel differential a poor yaw-rate estimator (section 3.6).

Table 3.2: Simulated sensor configuration. Intrinsic are analytic rather than calibrated: the simulator renders an exact pinhole camera, so $f_x = f_y$ follow from the image width and the horizontal field of view and the distortion coefficients are identically zero. The stereo baseline was independently checked from positive disparity in a recorded bag rather than taken from the model alone.

Quantity	cam0 (left)	cam1 (right)	imu
Resolution	1280 × 720	1280 × 720	—
Rate	30 Hz	30 Hz	200 Hz
Horizontal FOV	$\pi/2$ rad (90°)	$\pi/2$ rad (90°)	—
$f_x = f_y$	640	640	—
(c_x, c_y)	(640, 360)	(640, 360)	—
Distortion	zero	zero	—
Position from base_footprint (m)	(0.496, 0.0936, 0.4395)	(0.496, 0, 0.4395)	(0.338, 0, 0.192)
Position from IMU body (m)	(0.158, 0.0936, 0.2475)	(0.158, 0, 0.2475)	(0, 0, 0)
Stereo baseline	0.0936 m		—

The fixed camera rotation converts the ROS body convention (forward, left, up) to the optical convention (right, down, forward). The VINS simulation configuration fixes the extrinsics (`estimate_extrinsic: 0`) and disables temporal-offset estimation (`estimate_td: 0`); the assumed inertial noise model is given in table 3.3, and the same values are mapped to the ORB-SLAM3 configuration.

[Queried] The configured gyroscope noise density is 2×10^{-3} whereas the stationary windows of the five recorded streams give 3.00×10^{-4} to 3.64×10^{-4} , so the configured value is about six times too large. The configured accelerometer density of 0.02 sits inside the measured spread but that spread is itself wide, 0.0139 to 0.0294 across five bags of the same simulated sensor. Both estimators are therefore running on an inertial noise model that does not describe this IMU closely. The bias random walks cannot be checked

Table 3.3: Inertial noise parameters supplied to the visual-inertial estimators, beside the values measured from the recorded IMU stream in [section 3.6](#). The configured values are inherited estimator defaults, not measurements of this simulated IMU; the discrepancy is carried forward as a queried item rather than silently reconciled.

Parameter	Configured (VINS, ORB)	Measured from bag	Units
<code>acc_n</code>	0.02	0.0139–0.0294	$\text{m s}^{-2}/\sqrt{\text{Hz}}$
<code>gyr_n</code>	0.002	0.000300–0.000364	$\text{rad s}^{-1}/\sqrt{\text{Hz}}$
<code>acc_w</code>	0.001	not measurable	$\text{m s}^{-3}/\sqrt{\text{Hz}}$
<code>gyr_w</code>	0.0001	not measurable	$\text{rad s}^{-2}/\sqrt{\text{Hz}}$

at all: the longest stationary interval in any `dataset_allsensor` bag is 1.54 s and an Allan-variance estimate needs minutes. No conclusion in this report depends on these four numbers, but they should be re-derived from a dedicated stationary recording before any tuning claim is made.

[Queried] The accelerometer characterisation is not stable across recordings of the same simulated sensor. `allsensor_002`, whose stationary window is 1.54 s and therefore the best conditioned of the three, returns an x bias of 0.0683 m s^{-2} , against 0.0191 for `allsensor_000` and 0.0016 m s^{-2} for `allsensor_004`, whose window is the shortest at 0.52 s. The scale ranges from 1.0199 to 1.0669 over the five. Because the window is short in every case, part of this spread is estimator variance rather than a property of the sensor, and the accelerometer terms should not be quoted as calibrated constants. The gyroscope scale agrees to four decimal places across all five bags; its bias does not, varying by a factor of twenty, which is consistent with the same window-length problem.

Table 3.4: Platform geometry and drivetrain limits used by the wheel-odometry and EKF baselines. `AckermannSteering` exposes no noise parameter, so the simulated encoders and odometry are noise free at source.

Quantity	Symbol	Value	Source field
Wheel base (front to rear axle)	L	0.42 m	<code>wheel_base</code>
Track (wheel separation)	B	0.42 m	<code>wheel_separation</code>
Kingpin width	–	0.42 m	<code>kingpin_width</code>
Wheel radius	r	0.0585 m	<code>wheel_radius</code>
Wheel width	–	0.065 m	<code>wheel <length></code>
Steering limit	$\delta_{\max}$	$\pm 0.6109 \text{ rad}$	<code>steering_limit</code>
Chassis mass	–	10.1 kg	<code>base_footprint <mass></code>
Maximum speed	–	10 m s^{-1}	<code>max_velocity</code>
Maximum acceleration	–	3 m s^{-2}	<code>max_acceleration</code>
Odometry publish rate	–	50 Hz	<code>odom_publish_frequency</code>
Joint-state publish rate	–	1 kHz	measured from bag

3.2.3 Simulation fidelity and its limits

Every simulation result in this report rests on how faithfully the simulator reproduces the sensors and the contact physics. This subsection states what is modelled, what is idealised,

and in which direction each idealisation biases the comparison between the visual and proprioceptive estimators.

Inertial measurement. Table 3.3 reports the noise measured from the recorded stream and queries it, because no `dataset_allsensor` bag contains a stationary interval long enough for a sound estimate. That query can be closed without a new recording: the noise is *declared*, not emergent, so the simulator’s own configuration is the ground truth the measurement was trying to recover.

The `<imu>` sensor block injects zero-mean Gaussian noise at 200 Hz with per-sample standard deviations of $0.0048 \text{ rad s}^{-1}$ and 0.0566 m s^{-2} — densities of 3.394×10^{-4} and 4.002×10^{-3} in their respective units per $\sqrt{\text{Hz}}$. The measured gyroscope range of 3.00×10^{-4} to 3.64×10^{-4} brackets the declared value, which corroborates both. The measured accelerometer range of 1.39×10^{-2} to 2.94×10^{-2} does not: it is three to seven times the declared figure, and the discrepancy is the estimator’s, not the sensor’s, since the residual on the driving axis carries real longitudinal acceleration that a 1.54 s window cannot separate from noise. The declared values should be preferred wherever a number is needed.

Against the ADIS16448 of the EuRoC benchmark, on which both estimators were published, the simulated unit is *twice as noisy* on both channels, so it is not an optimistic sensor. Bias is first-order Gauss–Markov with correlation times of 1000 s and 300 s.

The IMU is therefore not optimistic in its noise. Three things are nonetheless absent: chassis vibration, which on a real platform is frequently the dominant error; scale-factor, cross-axis and g -sensitivity errors; and temperature drift. The correlation times also exceed the 85 s to 130 s run lengths, so within a run the bias behaves as a slowly varying constant and its instability is never exercised. Finally, the message’s `orientation` field carries no noise block at all and is therefore ground truth; no estimator in this project consumes it, and none should.

Wheel encoders. `JointStatePublisher` reports the true solver angle. Slip consequently appears as a *discrepancy* rather than an error — the joint angle stays true, so encoder-integrated distance over-reads whenever traction breaks, which is exactly a real encoder’s behaviour. What is idealised is resolution: a physical 1024 CPR encoder quantises to 6.1×10^{-3} rad, setting a floor on low-speed velocity estimation that the simulation does not have. `script/encoder_sim.py` exists to impose that floor downstream and has not been applied to any result in this report.

Contact friction, and why the encoder is flattered. ODE uses the smaller friction coefficient of the two contacting surfaces. The wheels declare $\mu = 1$, $\mu_2 = 1$; every ground model declares $\mu = 100$, $\mu_2 = 50$; so the contact runs at $\mu = 1.0$, the optimistic end of the published dry-clean-concrete range. Force-dependent slip is disabled, `slip1` and `slip2` being pinned at zero on all three ground models, so contact is rigid until Coulomb traction is exceeded.

Table 3.5 gives the consequence, measured rather than assumed, by integrating rear-wheel angle against true path length.

Slip is present but under 1 % everywhere, and on `allsensor_001` the encoder under-reads, which cannot be slip. Against a wheel-odometry drift of 2.15 % to 6.57 %, slip is therefore not the dominant error; the yaw-rate geometry is, as the falsification of H1 establishes.

Table 3.5: Measured wheel slip on the five multi-sensor recordings. The ratio is encoder-integrated distance over true path length; above one is slip, below one is a rolling-radius error and cannot be slip. Measured by `script/check_slip.py`.

Recording	Encoder (m)	Truth (m)	Ratio
<code>allsensor_000</code>	117.12	117.05	1.0006
<code>allsensor_001</code>	112.91	113.11	0.9982
<code>allsensor_002</code>	115.37	115.20	1.0015
<code>allsensor_003</code>	141.68	140.61	1.0076
<code>allsensor_004</code>	142.36	142.25	1.0008

[Threat to validity] This is a bias with a known direction. The EKF baseline of section 4.2 is evaluated on a floor where the wheels effectively cannot slip and with an encoder of unlimited resolution, which is dead reckoning’s best case. Both idealisations favour the EKF over the visual estimators, and neither has been relaxed. The conclusion that a wheel+IMU filter outperforms both visual pipelines should be read as holding *under these conditions*, not as a general result. `script/make_slippery_floor.py` generates ground models at a chosen friction coefficient so that traction can be made an experimental factor; `script/encoder_sim.py` imposes a realistic quantisation floor. Neither has been used in any result reported here.

Noise models supplied to the estimators. The two are not told the truth about the IMU. Both configurations set `acc_n` = 0.02 and `gyr_n` = 0.002, over-stating the injected densities by 5.0 and 5.9 times respectively, and `gyr_w` by a factor of 58. Inflating noise above datasheet values is standard practice in visual-inertial odometry, and the configuration records that intent. The EKF baseline, by contrast, derives its noise model by measuring it against ground truth from the same recording.

That asymmetry was tested directly rather than left as a caveat. Configurations carrying the true injected densities — `stereo_imu_truenoise.yaml` for both estimators — were run on all five recordings on 18 September 2026, after the reporting cutoff. Accuracy improved wherever a run completed: VINS-Fusion on four of five, by 43 % and 32 % on two of them. But three of ten runs diverged outright, reaching 111 m to 3899 m of error with 516 to 1545 tracking jumps, where the inflated configuration diverged in none. The behaviour is bimodal rather than graded: surviving runs carry zero to three jumps, failed runs hundreds. This confirms the configuration comment’s own claim that too-large noise degrades gracefully while too-small does not, and the inflated setting is retained for every result in this report. These runs are recorded as `logging_20260916_03` and are not part of the reported campaign.

3.2.4 Real sensor rig and calibration limits

The real configuration is summarised in table 3.6. Its calibration is materially weaker than the simulated rig’s, and the weaknesses are load bearing rather than cosmetic.

The real IMU-to-camera rotation has observational support, but its translation is a zero placeholder. The stationary accelerometer magnitude is recorded as 9.2642 m s^{-2} , rather than 9.81 m s^{-2} . VINS-Fusion uses the observed magnitude as `g_norm`; the ORB-SLAM3 real inertial launch rescales acceleration by $9.81/9.2642$. These treatments make the configurations internally consistent but do not replace an independent camera–IMU

Table 3.6: Real sensor rig, and the calibration status of each quantity. Only the first three rows are measured; the remainder are approximations or placeholders that bound what can be claimed from real-robot data.

Quantity	Value	Status
Stereo resolution	1920 × 1080 fisheye	measured
Stereo baseline	0.093 604 m	measured
IMU stream	/hwt101ct_yaw_publisher	separate RealSense source
Camera model (ORB-SLAM3)	KannalaBrandt8	configured
Camera model (VINS)	project fisheye files	configured
Second-camera intrinsics	reuse of the first camera	approximation
IMU-to-camera rotation	observational support	partial
IMU-to-camera translation	zero	placeholder
Stationary $\ a\ $	9.2642 m s ⁻²	measured, deviates from 9.81
Extrinsic estimation (VINS)	estimate_extrinsic: 1	online
Time-offset estimation (VINS)	estimate_td: 1	online

calibration. Real trajectories can be inspected for plausibility and tracking continuity, but metric accuracy cannot be claimed without independent ground truth and improved extrinsic calibration.

3.2.5 Coordinate-frame contract

Standalone VINS-Fusion publishes `world` → `body`. Standalone ORB-SLAM3 publishes an estimator-specific world frame. For RTAB-Map integration, both publish `odom` → `vio_body`; a measured static transform connects `vio_body` → `base_footprint` by (−0.338, 0, −0.192) m. RTAB-Map then publishes the external correction `map` → `odom` (figure 3.3).

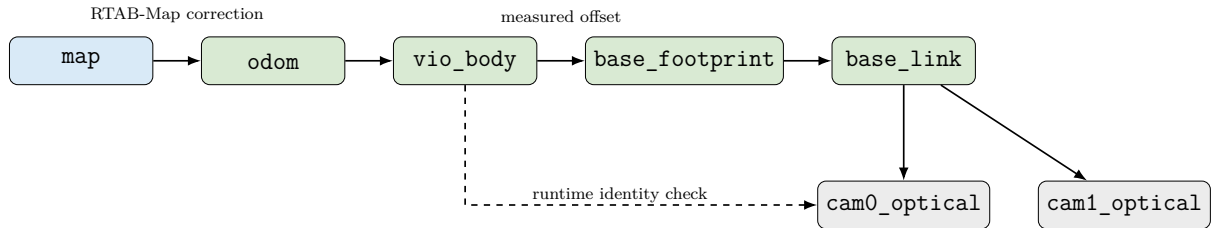


Figure 3.3: Coordinate-frame contract used by the composed RTAB-Map pipeline. The dashed connection is a calibration consistency check, not a second pose publisher.

The two independent paths to `cam0_optical` provide a runtime calibration check: `tf2_echo cam0_undis cam0_optical` must return zero translation and the identity quaternion. This check has been measured successfully. It is needed because an earlier OpenCV YAML parsing error yielded a plausible translation but a non-rigid rotation and invalidated a map-quality campaign.

The standalone simulation ORB launch still labels the estimated IMU-body pose as `base_footprint`. The composing RTAB-Map launch corrects this by using `vio_body`;

standalone consumers must not assume that the default label is the chassis origin. Constant offsets are mostly absorbed by rigid trajectory alignment but remain material for map placement and robot-footprint tests.

3.3 Environments, Datasets, and Controlled Factors

3.3.1 Simulation environment

The principal large warehouse is a 41.2833 m square environment produced by rescaling the AWS warehouse to 1704 m², while retaining a 9.020 m interior height. Fourteen marked storage bays and five reduced-width walkways are represented. Static and dynamic families are stored under `worlds/small_warehouse_static/` and `worlds/small_warehouse_dynamic/`. The original tiny warehouse under `worlds/small_warehouse/` is a distinct environment; tiny- and large-layout results are not repeated trials of the same condition.

The occupancy truth of the object arrangements is shown in [figure 3.4](#). These grids are not SLAM products: they are rasterised directly from the world’s collision meshes by `script/bake_map.py`, sliced at robot height, so they contain no unknown space and regenerate whenever the world changes. They are the reference against which any reconstructed map is scored.

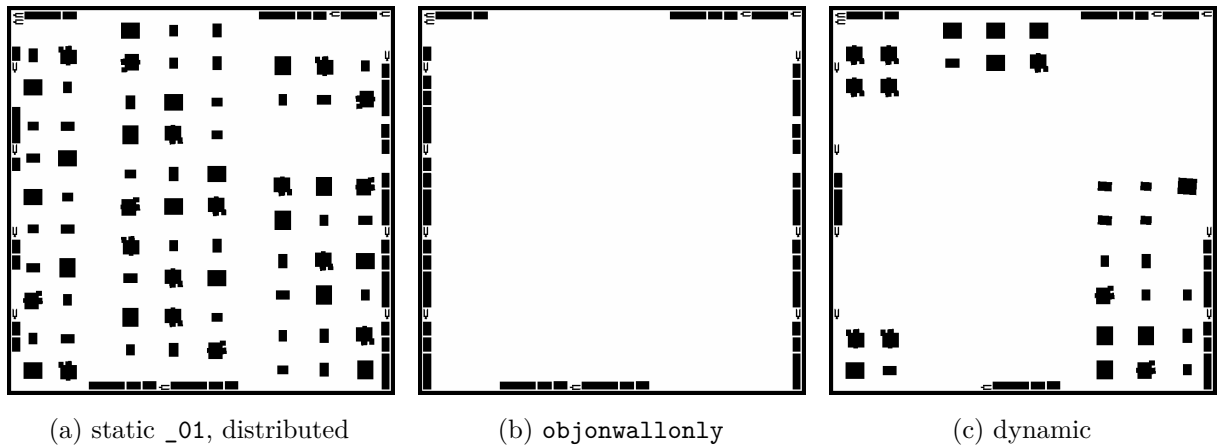


Figure 3.4: Baked occupancy truth of the three large-warehouse object arrangements, each 42.2 m square at 0.05 m per cell with origin $(-21.1, -21.1)$ m. Black is occupied, white is free; there is no unknown space because the grids are rasterised from collision geometry rather than observed. Occupied cells are 17.79 % of (a), 8.74 % of (b) and 10.98 % of (c). Panel (b) clears the central floor and keeps obstacles against the walls, which is the arrangement used by `dataset_allsensor_002` and `dataset_allsensor_003`. *No occupancy grid is shown per floor treatment, and none exists*: the `bayline` and plain floors differ only in surface material, which carries no collision geometry, so their baked grids are identical to the panels above. See [figure 3.5](#).

The floor treatment is a separate factor and is invisible to occupancy. It is set by which ground model a world includes; those models share one mesh and differ only in the visual materials attached to it ([table 3.7](#)). [Figure 3.5](#) renders both recorded conditions at true scale using `script/render_floor.py`, which drives the 2-D viewer’s own floor rasteriser so that the figure shows what the overlay and the simulated cameras see rather than a texture file out of context.

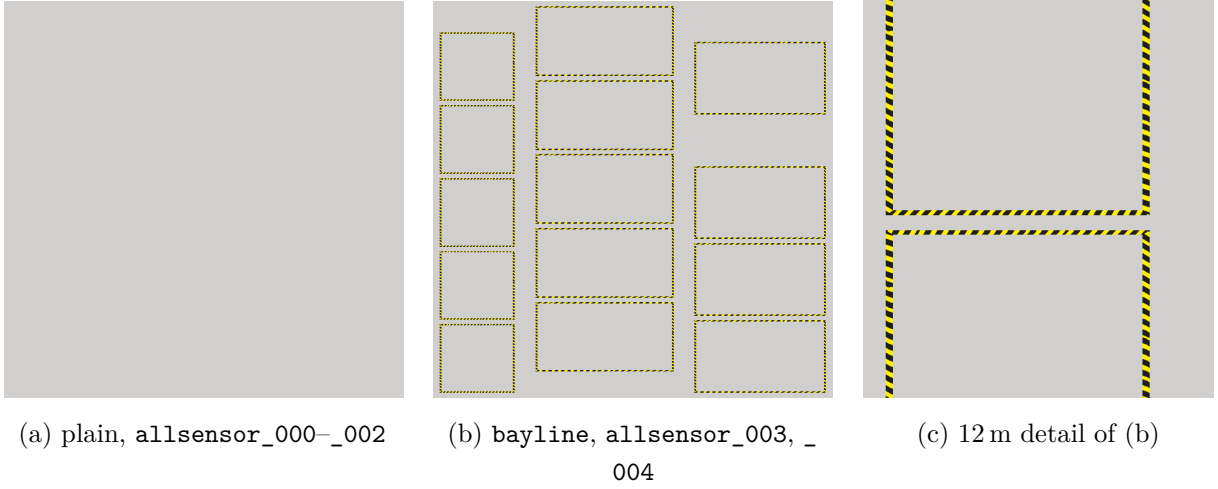


Figure 3.5: The two recorded floor conditions, rendered across the full 42.2 m floor at 0.05 m per pixel. Both share the same flat diffuse base, RGB (0.814, 0.811, 0.804); the only difference between them is the 112 painted marking faces that **bayline** adds, forming the hazard-striped outlines of fourteen storage bays. Panel (c) is a 12 m square detail showing the strips at 0.145 m width. Everything not outlined is untextured in both conditions.

Table 3.7: Ground models and the floor conditions they produce. The mesh and its collision geometry are identical in all three, so the floor treatment changes what a camera sees and nothing a wheel, an occupancy grid or a collision check responds to. The concrete-grain texture belongs to the stock model only, and no recorded dataset uses it.

Ground model	Base	Concrete	Markings	Used by
..._GroundB_01 (stock)	texture	yes	112 faces	no recorded dataset
..._GroundB_01_bayline	flat diffuse	no	112 faces	allsensor_003, allsensor_004
..._GroundB_01_plain	flat diffuse	no	none	allsensor_000–_002, all nofloortexture datasets

[**Naming**] The factor recorded in this project as “textured versus featureless floor” is narrower than the name suggests, and [figure 3.5](#) is the evidence. Both conditions rest on the *same flat, untextured diffuse colour*; the stock concrete-grain texture appears in neither. The entire difference is fourteen painted bay outlines, 0.145 m wide, covering a small fraction of a 1704 m² floor. A robot driving an aisle sees a uniform grey surface in both conditions for most of its path, crossing a high-contrast strip only at a bay boundary.

Results comparing the two floors must therefore be read as *bay markings versus nothing*, not as *realistic floor versus nothing*, and the near-absence of a visual-odometry improvement between them ([section 4.2](#)) should not be generalised to floors carrying genuine surface texture. Testing that would require a world built on the stock `GroundB_01` model, which no recorded dataset uses.

[**Unverified**] The dynamic world’s baked grid, [figure 3.4\(c\)](#), contains 38 % fewer occupied cells than the distributed static arrangement in (a). `bake_map.py` rasterises every `<include>` in the world file at its authored pose, so kinematically driven props should appear at their initial positions, but this has not been confirmed for the dynamic family. Until it is, panel (c) must not be used as occupancy truth for scoring a reconstructed map of a dynamic scene: the deficit may be a genuine difference in static stocking, or it may be props missing from the bake, and those two possibilities imply opposite corrections. The estimator results in [section 4.2](#) are unaffected, as they use the arrangements in panels (a) and (b) only.

The available controlled factors are summarised in [table 3.8](#).

Table 3.8: Controlled experimental factors available in the simulation environment, with the levels realised at the reporting cutoff. A factor is listed as exercised only where a recorded dataset exists for more than one of its levels.

Factor	Levels	Status at cutoff
Scene dynamics	static props; scripted dynamic props	exercised
Floor markings	bay outlines present (<code>bayline</code>); absent (plain)	exercised; no dataset carries surface texture
Prop or route arrangement	canonical; <code>_00</code> ; <code>_01</code> ; <code>_02</code>	exercised
Stocking level	partial; fully stocked static	fully stocked recorded
Object placement	distributed; restricted to the wall region	exercised, three bags to one
Domain	simulation; real robot	exercised, no real ground truth
World scale	tiny <code>small_warehouse</code> ; large rescaled families	distinct environments, never pooled

The plain floor removes the painted bay outlines; neither recorded condition carries the stock concrete texture. Collision geometry and occupancy truth are preserved in both ([table 3.7](#)). Dynamic props follow deterministic routes through a kinematic Gazebo plugin. This preserves repeatable motion and acceptable real-time factor, but it does not reproduce physical collision response if the robot overlaps a moving prop.

World identity is recorded by exact `.world` path. ROS launches require `colcon build --symlink-install`; a plain build can leave an obsolete copied world in `install/`. When object motion is required by the 2-D viewer, model poses must also be bridged from the simulator.

3.3.2 Dataset registry

Datasets are held in two places and the registry must cover both: the workspace tree `dataset/` and a removable drive mounted at `/media/ambushee/32E4AAB1E4AA772F/`, whose simulation bags sit under `dataset/` and whose real-robot recordings sit at its root. Table 3.9 lists all eighteen recordings across both locations. Six exist in one place only on the drive, seven in the workspace only, and three are held in both.

The seven featureless-floor simulation bags of the relogged campaign (section 4.1) are the static and dynamic families listed above; five were read from the drive and two from the workspace, for the reason recorded next.

[Data integrity] Two drive copies are unreadable while presenting a complete, plausible `metadata.yaml`. `dataset_static_nofloortexture_001` is a 524 288 B stub whose metadata nevertheless declares 113 332 messages, and `dataset_dynamic_nofloortexture_01_000` is 710 MB—within 0.4 % of its workspace counterpart—with a valid SQLite header but a store that reports *database disk image is malformed* on any query. Both have intact workspace copies, which is why those two datasets were read from the workspace in the relogged campaign.

The general lesson is recorded as a control rather than an incident: `metadata.yaml` is written when recording begins and does not describe what survived, so message counts, durations and topic lists copied from it are not evidence that a bag is intact. Storage health must be established by opening the SQLite store and counting rows, which is now part of the pre-experiment inspection in section 3.8.

[Scope limit] Only the five `dataset_allsensor` bags carry wheel encoders and wheel odometry; the other thirteen recordings omit those topics entirely. The proprioceptive baseline of section 4.2 therefore cannot be computed for any dynamic condition, for the mapping bags, or for the real rig, and the comparison between proprioception and visual odometry exists only in *static, featureless-floor* scenes—the conditions least favourable to vision and most favourable to dead reckoning. Extending it requires re-recording with the two topics bridged, not new analysis.

[Excluded] `dataset_real_001` and `dataset_real_002` are excluded because their inertial topic `/hwt101ct_yaw_publisher` contains *zero* messages, so neither supports stereo-inertial operation at all. Their image streams are intact (2 562 and 3 405 pairs), so they remain usable for detector or image-only work. `dataset_real_000` is the only real recording with inertial data, at 23 257 messages.

Historical output folders do not prove that their raw bags remain available or valid. Conversely, a dataset is not marked broken merely because its only copy lives on the removable drive rather than in the repository; six of the fifteen recordings are drive-only and are sound.

Before a new experiment, the bag’s absolute path, topics, duration, message counts, and storage health are inspected. Each paired comparison uses the same recording, and estimators run separately to avoid resource-contention effects. Environment, trajectory, calibration, replay rate, image transport, initialization treatment, and revision are controlled. A run is excluded if input overlap, initialization, processed sample count, or sensor delivery differs materially from its paired condition.

Recorded simulation bags used by RTAB-Map contain `/clock`. They are played without rosbag’s additional `-clock` option because competing clock sources cause TF time jumps. This choice is made after inspecting each bag, not applied as a universal command rule.

Table 3.9: Complete dataset registry at the reporting cutoff, across both storage locations. The bayline families carry a textured floor; every other simulation entry is featureless. “WS” is the workspace tree `dataset/` and “FD” the removable drive; a size is given where that location holds a copy. “Prop.” marks the proprioceptive group (`/model/ackermann_robot_001/joint_state` and `/model/ackermann_robot_001/odometry`); “IMU” counts inertial messages, on `/imu` for simulation and `/hwt101ct_yaw_publisher` for the real rig. Durations are rosbag wall-clock recording durations: the simulator ran below real time, so a bag’s sim-time span is shorter than the figure shown. Integrity was checked by opening each SQLite store and counting rows against the count its `metadata.yaml` declares, not by trusting the `metadata`.

Dataset	Family	WS (MB)	FD (MB)	Dur. (s)	Messages	/clock	Truth	Prop.	Integrity
<code>dataset_allsensor_000</code>	<code>static_01</code>	689	–	129.3	211 811	✓	✓	✓	ok
<code>dataset_allsensor_001</code>	<code>static_01</code>	674	–	126.2	198 873	✓	✓	✓	ok
<code>dataset_allsensor_002</code>	<code>static, wall only</code>	750	–	105.5	215 528	✓	✓	✓	ok
<code>dataset_allsensor_003</code>	<code>bayline, wall only</code>	1056	–	141.2	255 699	✓	✓	✓	ok
<code>dataset_allsensor_004</code>	<code>bayline, distributed</code>	1311	–	168.6	253 942	✓	✓	✓	ok
<code>dataset_static_nofloortexture_000</code>	<code>static</code>	–	592	101.5	118 597	✓	✓	–	ok
<code>dataset_static_nofloortexture_001</code>	<code>static</code>	617	1	92.0	113 332	✓	✓	–	FD stub, WS ok
<code>dataset_static_nofloortexture_002</code>	<code>static</code>	–	688	103.3	122 386	✓	✓	–	ok
<code>dataset_static_nofloortexture_objonwallonly</code>	<code>static, wall only</code>	648	–	110.5	114 523	✓	✓	–	ok
<code>dataset_dynamic_nofloortexture_00_000</code>	<code>dynamic_00</code>	–	653	100.3	111 988	✓	✓	–	ok
<code>dataset_dynamic_nofloortexture_00_001</code>	<code>dynamic_00</code>	–	778	167.7	114 169	✓	✓	–	ok
<code>dataset_dynamic_nofloortexture_01_000</code>	<code>dynamic_01</code>	713	710	165.4	126 790	✓	✓	–	FD malformed, WS ok
<code>dataset_dynamic_nofloortexture_02_000</code>	<code>dynamic_02</code>	–	758	172.1	134 995	✓	✓	–	ok
<code>dataset_map_01</code>	<code>mapping, large</code>	818	–	193.5	152 291	✓	✓	–	ok
<code>dataset_map_tiny</code>	<code>mapping, tiny world</code>	726	–	63.3	82 555	✓	✓	–	ok
<code>dataset_real_000</code>	<code>real rig</code>	2964	2964	116.6	36 166	–	–	–	ok, 23 257 IMU
<code>dataset_real_001</code>	<code>real rig</code>	–	2099	85.5	9 629	–	–	–	0 IMU messages
<code>dataset_real_002</code>	<code>real rig</code>	–	2869	113.5	12 682	–	–	–	0 IMU messages

3.4 ORB-SLAM3 and Semantic Feature Filtering

The project provides a purpose-built ROS 2 Humble wrapper around the pinned ORB-SLAM3 core. Approximate-time synchronization pairs left and right images with a 0.02 s slop. Raw and compressed transports are supported. Image callbacks place stereo pairs in a one-frame queue; a worker thread calls `TrackStereo()` so blocking visual tracking does not starve the 200 Hz IMU subscription. If tracking falls behind, the older pending image is discarded and the drop count is reported.

With `filter:=false`, no detection subscription is created and an empty mask passes through the patched interfaces. With filtering enabled, a bounded buffer is searched for the detection closest in source-image time, accepted only within 0.15 s. Boxes are expanded by eight pixels, clipped to the image, and rasterized to an 8-bit mask in which 255 retains a feature and zero rejects it. A mask covering more than 0.8 of the frame is discarded as a safety measure.

The mask passes through `System`, `Tracking`, and `Frame` to the ORB extractor. Candidate keypoints are tested before octree distribution, so the per-level feature quota can be redistributed across surviving static regions. At each pyramid level, the implemented boundary rule rejects a candidate when at least five samples in a 3×3 neighbourhood are dynamic. Only left-camera features are masked because ORB-SLAM3 establishes stereo matches from left keypoints to right candidates.

The current simulation YAML requests 1,000 ORB features, eight pyramid levels, a 1.2 scale factor, initial FAST threshold 20, and fallback threshold 7. An older engineering note states 1,500 features, but the current configuration is authoritative. No controlled feature-count or FAST-threshold sweep is complete.

The wrapper publishes `/orbslam3/odometry`, `/orbslam3/pose`, and TF after valid tracking. It writes the common 11-column trajectory format directly and flushes each row. Repository evidence is located in `orbslam3_ros2/src/orbslam3_node.cpp`, `orbslam3_ros2/src/slam_wrapper.cpp`, `orbslam3_ros2/src/trajectory_writer.cpp`, and `orbslam3_ros2/config/wil_sim/stereo_imu.yaml`.

3.5 VINS-Fusion and the RTAB-Map Back End

3.5.1 VINS-Fusion front end

VINS-Fusion uses stereo features and inertial measurements in a sliding-window nonlinear estimator. The ROS wrapper synchronizes the raw image topics and uses separate callback groups for images, IMU, feature messages, and detections. The simulation configuration processes every image (`image_skip: 1`), uses two cameras and IMU, fixes simulated extrinsics, targets 25 Hz feature publication, and uses 10-pixel keyframe parallax. Image queues were increased to 100 and the IMU queue to 2,000 to reduce losses during initialization and backlog.

VINS cannot use only the ORB strategy because it propagates existing features with pyramidal KLT optical flow and detects new features only for replenishment. A feature attached to an object before detection could otherwise persist after masking. The implementation initializes `FeatureTracker::setMask()` from the YOLO mask. Tracks in zero-valued regions fail the retention test, while the same mask is passed to `goodFeaturesToTrack()` to suppress new points. The count of persistent tracks removed in dynamic regions is accumulated as a mechanism diagnostic.

VINS writes the optimized state after each estimator update to `vio.csv` and broadcasts `world` $\rightarrow$ `body` using configurable frame names. The ROS `output_path` parameter overrides the YAML path, recursively creates missing directories, and reports an explicit error if `vio.csv` cannot be opened. This behaviour is an uncommitted worktree change at the cut-off. Evidence is in `vins_fusion_ros2/src/vins_estimator.cpp`, `vins_fusion_ros2/vins/src/featureTracker/feature_tracker.cpp`, and `vins_fusion_ros2/vins/src/estimator/estimator.cpp`.

3.5.2 RTAB-Map mapping and localization back end

RTAB-Map receives front-end odometry through TF and stereo imagery through `rtab_camera_shim.py`. The shim reconstructs `CameraInfo`, decompresses the pair, and publishes validated static camera transforms from the common simulation calibration. TF was selected over an odometry topic because adding high-rate odometry to multi-topic approximate synchronization starved the lower-rate images. The cost is that odometry covariance is not supplied to graph edge weighting.

The back end uses stereo SGBM (`Stereo/DenseStrategy: 1`), height-based ground separation, ray tracing, spatial noise filtering, and odometry-derived gravity links. Its measured default depth range is 10 m. Because the featureless floor provides essentially no stereo ground points, ray tracing supplies free-space evidence between the sensor and observed obstacles. These settings affect occupancy assembly; visual-word loop closure is separate.

Three modes are provided by one launch file:

construct starts a new incremental database;

extend opens an existing database, places prior nodes in working memory, and permits new nodes; and

localize disables incremental memory, loads existing nodes, serves the saved map, and can enable `Mem/LocalizationReadOnly`.

Localization still extracts features from current stereo images and matches stored RTAB-Map signatures. Read-only mode prevents database modification; it does not disable perception. The outputs are an external `map` $\rightarrow$ `odom` correction and `/localization_pose`. Neither RTAB-Map features nor corrected poses are injected into ORB-SLAM3's Atlas or VINS-Fusion's sliding window.

The composed localization launch raises `RGBD/OptimizeMaxError` from 3.0 to 10.0 because the default rejected all loop closures proposed with the observed VINS drift. This is effective operationally but is not yet proven safe against perceptual aliasing in repeated warehouse bays. Occupancy creation is disabled in localization mode because the stored map is reused. Live VINS-Fusion and RTAB-Map co-execution nevertheless remains blocked on the current machine. Recorded `vio.csv` can be replayed as `odom` $\rightarrow$ `vio_body` for a deterministic back-end test, but this is not proof of real-time co-execution.

Evidence is in `rtabmap_ros/rtab_sim.launch.py`, `rtabmap_ros/rtab_camera_shim.py`, `aws-robomaker-small-warehouse-world/launch/localization_rtabmap.launch.py`, and `script/gt_tf_publisher.py`.

3.6 Non-Visual Baseline Estimators

The visual estimators were, until the cutoff, scored only against ground truth. That establishes how large their error is but not whether it is good, because no alternative was

measured on the same data. Three non-visual estimators were therefore implemented in `script/proprio_estimator.py` to supply a dead-reckoning floor: wheel odometry, IMU dead reckoning, and an extended Kalman filter fusing the two. They consume only proprioceptive topics and no images.

3.6.1 Implementation form and why it is offline

All three read the recorded bag directly and are deterministic functions of it. Run as live ROS nodes they would instead depend on replay rate, QoS delivery and CPU contention, none of which is a property of the estimator being compared. The cost is a stated asymmetry: the visual figures they are compared against were produced by live runs, so any timing-induced error is present on one side of the comparison only. This is revisited in [section 4.2](#).

3.6.2 Sensor model derived from data

Every covariance field in the source bags is identically zero—0 of 17 416 IMU messages, 0 of 4 355 wheel-odometry messages, and 0 of 4 354 ground-truth messages in `dataset_allsensor_000`. Gazebo Fortress does not populate them, and `AckermannSteering` exposes no noise parameter. The noise model is consequently measured rather than read, by a `calibrate` step that writes `sensor_noise.yaml`; [table 3.10](#) gives the result for both bags.

Table 3.10: Sensor noise and scale measured from each bag by `proprio_estimator.py` `calibrate`. Scales are least-squares gains through the origin against ground truth; σ values are the residual standard deviations. A scale of 1.0 is an unbiased sensor. The bias random walks are assumed, not measured, because the longest stationary window is 1.54 s.

Group	Quantity	_000 plain	_001 plain	_002 plain	_003 bayline	_004 bayline	Units
IMU	gyroscope z bias	6.48×10^{-5}	7.75×10^{-5}	9.28×10^{-5}	6.09×10^{-4}	2.58×10^{-5}	rad s^{-1}
	gyroscope σ	4.36×10^{-3}	4.24×10^{-3}	4.81×10^{-3}	5.03×10^{-3}	5.15×10^{-3}	rad s^{-1}
	gyroscope z scale	0.9990	1.0000	1.0002	1.0002	1.0000	—
	accelerometer x bias	0.0191	0.0304	0.0683	0.0309	0.0016	m s^{-2}
	accelerometer x scale	1.0199	1.0249	1.0669	1.0525	1.0465	—
	measured g	9.7809	9.8191	9.7827	9.8037	9.8346	m s^{-2}
Wheel	speed scale	1.0002	0.9985	1.0005	1.0071	1.0001	—
	speed σ	0.0407	0.0484	0.0496	0.0607	0.0505	m s^{-1}
	yaw-rate scale, bicycle	1.0110	0.9527	1.0034	0.9375	0.9250	—
	yaw-rate σ , bicycle	0.0659	0.0804	0.0705	0.1192	0.0613	rad s^{-1}
	yaw-rate scale, differential	1.2162	1.1442	1.2194	1.2748	1.1404	—
	yaw-rate σ , differential	0.2359	0.3074	0.2620	0.3477	0.2828	rad s^{-1}
Window	static duration	0.68	0.58	1.54	1.04	0.52	s
	maximum tilt	0.0016	0.0018	0.0015	0.0018	0.0017	°

Two consequences follow directly. First, wheel *speed* is essentially unbiased on all five bags (scale 1.0002 to 1.0071), so distance travelled is not the error source. Second, the rear-wheel differential is a far worse yaw-rate estimator than the steering-angle bicycle model—residuals of 0.26 rad s^{-1} to 0.35 rad s^{-1} against 0.061 to 0.119—because the track and the wheel base are both 0.42 m and a solid rear axle slips through every turn. The bicycle model is selected automatically and was chosen on all five bags.

3.6.3 The orientation field must not be used

`sensor_msgs/Imu.orientation` in these bags is not an estimate. Measured over `dataset_allsensor_000` it equals ground-truth yaw minus exactly 90° , with residual standard deviation $0.000\,00^\circ$ and $0.000\,00^\circ$ of drift across 87s: the plugin publishes the true orientation from the simulator’s entity-component manager. Consuming it would give a “dead-reckoning” estimator a perfect attitude reference that no real IMU has, and its output would measure ground truth. All three estimators therefore integrate `angular_velocity.z` instead, which drifts honestly by 2.52° over 87s on `allsensor_000` and 1.70° over 81.8s on `allsensor_001`. The same check on `allsensor_002` again returns a residual standard deviation of $0.000\,000^\circ$ against ground-truth yaw, so the exclusion is not specific to the first two recordings. Each run records `imu_orientation_used=NO` in its metadata so the exclusion is auditable.

3.6.4 Estimator definitions

Table 3.11 defines the three estimators. The EKF carries the planar state $\mathbf{x} = [p_x, p_y, \theta, v, b_\omega]^\top$, predicted at the 200 Hz IMU rate and corrected at 50 Hz by the wheels.

Table 3.11: Implemented non-visual estimators. “Anchoring” refers to the initial pose: dead reckoning has no global reference, so all three start from the ground-truth pose at their first sample, and this is recorded as an alignment decision rather than treated as neutral.

Estimator	Inputs	State	Correction
Wheel odometry	rear wheel speeds, steering angles (1 kHz)	$[p_x, p_y, \theta]$	none; $\mathbf{P}$ grows without bound
IMU dead reckoning	gyroscope z , accelerometer x (200 Hz)	$[p_x, p_y, \theta, v]$	none
EKF wheel + IMU	both of the above	$[p_x, p_y, \theta, v, b_\omega]$	wheel speed corrects v ; $(\omega_{\text{gyro}} - \omega_{\text{wheel}})$ corrects b_ω

The EKF’s second measurement is deliberately an observation of the *gyroscope bias* and not of heading. Integrating the wheel yaw rate into a heading and correcting θ with it would reproduce the wheel estimator inside the filter and inherit its unbounded drift. Treating the instantaneous rate difference as an observation of b_ω keeps each sensor supplying what it measures well: the wheels know rate, the gyroscope knows change, and neither knows absolute heading, so heading remains unobservable and its variance correctly continues to grow. The update uses the Joseph form, which keeps $\mathbf{P}$ symmetric and positive definite across the roughly 16 000 updates in a run.

Measured scale errors are recorded but never divided out. Correcting them would mean calibrating the estimator with the ground truth it is about to be scored against; the scale error is a real property of encoder odometry and is retained.

3.7 YOLO Model, Mask Construction, and Synchronization

The semantic front end is a standalone Python ROS 2 node using Ultralytics. The project checkpoint is `weight/best.pt`, a custom detector for `bin`, `box`, and `bucket`, fine-tuned from `yolo26m.pt` using 640-pixel inputs. It produces bounding boxes, not segmentation masks. The stock `yolo26n.pt` and `yolo26m.pt` files are not project-trained models and are not evidence of custom training.

At startup, the node resolves requested classes against checkpoint metadata, warms the model with three dummy images, and selects CUDA when available. Current defaults are confidence 0.35, NMS IoU 0.5, and image size 640. Only the left image is processed. Each `Detection2DArray` preserves the source image header, so nearest-timestamp selection measures sensor-time correspondence rather than inference-completion time. Empty arrays distinguish a valid frame with no detections from detector failure.

The detector intentionally remains outside stereo synchronization. A three-input image/image/detection synchronizer would discard pairs whenever inference lagged. The implemented design continues without filtering if no recent detection exists. Filtered runs write `yolo_mask.csv` with timestamp, box count, coverage, match state, detection age, and rejection status.

Offline evaluation in `script/yolo_eval.py` transforms Gazebo model poses and collision-mesh vertices into the left-camera frame to form projected truth boxes. Object speed over 0.05 m s^{-1} defines the moving subset. Predictions are greedily matched by class at IoU 0.5. Heavily occluded truth boxes and predictions covered by known but unlabeled box-like scenery are ignored at stated thresholds. Outputs are per-frame `yolo_eval.csv`, per-detection `yolo_detections.csv`, and overlays. Overlays are inspected before aggregate metrics because invalid pose stamps, centimetre mesh units, or malformed camera transforms can produce plausible but invalid numbers.

The current semantic rule assumes all selected-class detections are dynamic. No optical-flow, stereo-depth, or world-motion gate is implemented, and axis-aligned boxes also remove background pixels. These are explicit method limitations.

3.8 Collection Protocol, Validity, and Artifacts

The sensor source, estimator, optional detector, and viewer start as separate processes; launch files do not automatically play bags. Baseline and filtered treatments run separately on the same recording, at a replay rate that permits comparable input consumption. Competing estimators are not run simultaneously in a scientific A/B comparison.

The common trajectory file has no header and uses:

```
t_ns, px, py, pz, qw, qx, qy, qz, vx, vy, vz
```

Timestamps are integer nanoseconds from the source header and quaternion order is w, x, y, z . ORB-SLAM3 writes this representation directly; VINS writes the state after nonlinear optimization. Simulation truth is extracted to the same shape by `script/extract_gt.py`. The tracker-use record `yolo_mask.csv` remains separate from the offline detector score.

The current analysis code requires at least ten overlapping samples. Scientific acceptance additionally requires the same input interval, successful initialization, comparable pose

Table 3.12: Principal stored artifacts.

Artifact	Purpose
<code><run>/vio.csv</code>	Estimator trajectory and velocity
<code><run>/ground_truth.csv</code>	Simulation reference trajectory, when available
<code><run>/yolo_mask.csv</code>	Actual mask matching, coverage, and rejection
<code><run>/run_metadata.csv</code>	Resolved configuration and supplied provenance
<code><run>/performance.csv</code>	Per-frame timing, state, delivery, CPU, and memory
<code><run>/frontend.csv</code> and <code><run>/backend.csv</code>	VINS feature-tracking, estimator-stage, solver, and state-window diagnostics
<code><run>/tracking_frontend.csv</code>	ORB-SLAM3 tracking-stage timing and feature counts
<code><run>/local_mapping.csv</code>	ORB-SLAM3 keyframe insertion, map-point, and local-BA diagnostics
<code><run>/loop_closing.csv</code>	Place-recognition, loop, merge, and global-correction events
<code><run>/run_summary.csv</code>	Graceful-shutdown validity and robustness totals
<code><run>/events.csv</code>	VINS initialization, reset-cause, and shutdown events
<code>output/yolo_eval/<experiment>/yolo_eval.csv</code>	Per-frame detector evaluation
<code>output/yolo_eval/<experiment>/yolo_detections.csv</code>	Per-detection matching
<code>output/yolo_eval/<experiment>/overlay/</code>	Projection and matching validation
<code>map/*.db</code> or <code><run>/rtabmap.db</code>	Persistent RTAB-Map graph and signatures
<code>output/compare/<domain>/<experiment>/</code>	Derived figures and summaries

and frame counts, and no unexplained sensor starvation. Steps implying over 10 m s^{-1} are tracking jumps; an indoor estimate with error beyond 50 m is divergence rather than ordinary inaccuracy. Without real-data ground truth, path length and mean speed are plausibility diagnostics, not accuracy measurements.

The estimator now records resolved local configuration and optional experiment, dataset, world, replay-rate, command, and note fields in `run_metadata.csv`. It does not automatically determine bag health, repository revisions, complete dependency versions, external-process state, or the analyst’s exclusion decision. Empty optional fields therefore remain incomplete provenance. Historical folders created before this instrumentation remain preliminary unless their provenance is recoverable from logs and engineering records.

3.9 Evaluation Metrics and Analysis

3.9.1 Trajectory evaluation

`script/vio_metrics.py` associates estimate and reference over their common time interval and interpolates the reference at estimator timestamps. A rigid Umeyama $SE(3)$ fit maps estimated positions to the reference with scale fixed at one. Fixed scale is appropriate for stereo-inertial estimators and preserves scale error. Outputs include translational ATE RMSE, maximum and final error, rotational geodesic error, path length, overlap duration, pose count, jump count, and longest jump-free segment. Optional startup trimming is relative to each estimator’s first overlapping pose, preventing unequal initialization latency from being interpreted as steady-state behaviour.

The older `plot_vio_vs_gt.py` provides a first-pose-aligned diagnostic, but headline comparison uses common $SE(3)$ alignment. Real data without independent ground truth receives no ATE.

Relative pose error. RPE was added to `script/vio_metrics.py` after the reporting cutoff, on 18 September 2026, and is **implemented and exercised on the full campaign**. It follows the conventional definition: for a pair of poses separated by a fixed interval Δ , the error transform is

$$E_i = \left(Q_i^{-1}Q_{i+k}\right)^{-1} \left(P_i^{-1}P_{i+k}\right), \quad (3.1)$$

with P the estimate and Q the reference. Translational RPE is $\|\text{trans}(E_i)\|$ and rotational RPE is the geodesic angle of $\text{rot}(E_i)$, the same angle measure used for orientation error above.

RPE is reported because it needs *no* global alignment, and that is what makes it the appropriate metric for the dynamic runs. A single discontinuity re-seats the rigid $SE(3)$ fit, so every subsequent pose inherits the offset and ATE describes the jump rather than the tracking. RPE instead asks a local question — over the next Δ seconds, did the estimate move as the reference moved — and is insensitive to both the global frame and accumulated drift. It therefore supersedes the longest-jump-free-segment statistic, which answers a similar question through a segment-selection heuristic rather than a standard definition.

Four intervals are swept, $\Delta \in \{0.5, 1, 2, 5\}\text{ s}$, giving an error-versus-interval curve; $\Delta = 1\text{ s}$ is the headline value and corresponds to roughly 1.4 m at the simulated robot’s cruising speed. A pair is admitted only when its realised separation lies within 25 % of Δ ,

so a gap in the log cannot silently widen the interval and inflate the error; the surviving pair count is reported with every statistic. Pairs are overlapping, which is the conventional default.

Each statistic is computed twice: over all admissible pairs, and over only those pairs that do not span a detected tracking jump. The difference isolates the teleport contribution from ordinary local error, mirroring the existing separation between global and clean-segment ATE. Both RMSE and median are reported, because the dynamic-run distributions are strongly right-skewed and a single teleport otherwise sets the RMSE.

The implementation is verified against synthetic trajectories with known answers: an estimate identical to the reference returns zero; a constant relative drift of 0.1 m s^{-1} returns 0.100 m at $\Delta = 1 \text{ s}$ and 0.200 m at $\Delta = 2 \text{ s}$; a 2° s^{-1} relative rotation returns 2.00° at $\Delta = 1 \text{ s}$; and an injected teleport separates the all-pairs and jump-free statistics by the expected pair count. Results are written to `rpe_metrics.csv` for the interval sweep, with the $\Delta = 1 \text{ s}$ values carried into `trajectory_metrics.csv`.

3.9.2 Detection evaluation

Detector precision, recall, TP, FP, FN, moving/static recall, IoU, and image-area coverage are computed by `yolo_eval.py` at recorded confidence, matching-IoU, occlusion, minimum-size, and motion thresholds. Mask-use statistics are joined to trajectories by `timestamp_ns`. A semantic class is never treated as ground-truth motion.

3.9.3 Map and localization evaluation

RTAB-Map grids are placed in the Gazebo world using one fixed rigid world-to-map transform measured by `script/fit_map_pose.py`. It is constant for a database; the live `map` $\rightarrow$ `odom` drift correction is not reused for map placement.

`script/validate_map.py` rasterizes world collision geometry and evaluates occupied cells at a stated tolerance. It reports occupied-cell precision, whole-world and observed-region recall, false obstacles under the true $0.83 \times 0.50 \text{ m}$ footprint, unmapped footprint poses, and clearance. Graph evaluation records neighbour, global-closure, local-space, and gravity links. Localization additionally requires time to first fix, accepted and rejected matches, database checksums before and after read-only operation, pose continuity, and known-world bounds. Finite covariance or a localization status message alone does not establish correctness.

At the cutoff this evaluation has been applied to one database, `map/map_tiny.db`, against `small_warehouse_static/small_warehouse_static_tinyworld.world`. The equivalent large-warehouse evaluation is **planned** and no database for it exists; results are reported in [section 4.7](#).

3.9.4 Computational performance

Both estimator processes write per-frame `performance.csv` files. These record processing time, delivery and backlog counters, tracking or solver state, initialization and reset evidence, pose and keyframe accounting, cumulative process CPU time, monotonic wall time, and resident memory. VINS additionally records feature-front-end and estimator-back-end stages; ORB-SLAM3 records its tracking front end, local mapping, place recognition, loop closure, map merging, and global bundle-adjustment events. CPU utilization and

throughput can therefore be derived over explicit wall-time intervals. The measurements are process-local and exclude Gazebo, rosbag, RViz, RTAB-Map, and the separate YOLO process. Synchronized whole-system monitoring and the repeated Jetson/laptop matrix remain **planned**. Module timings from asynchronous stages are reported as separate distributions and are not added as if they formed one serial critical path.

3.10 Reproducibility Controls and Implementation Deviations

The following controls govern future experiments and acceptance of historical results:

1. Record absolute bag and exact `.world` file, treatment, configuration, commands, clock and replay rate, hardware, dependencies, and relevant repository commits.
2. Use a unique output directory and retain failed runs with failure labels.
3. Run paired treatments independently on the same sensor data and compare only their common valid interval.
4. Record inputs, outputs, initialization, dropped frames, resets, tracking loss, warnings, and exclusion reasons.
5. Validate camera order, transport, timestamps, metric scale, and every rigid transform before analysis.
6. Inspect representative detector and map overlays before aggregate scores.
7. Repeat nondeterministic RTAB-Map processing and report valid-run count; do not interpret differences within run-to-run variation as algorithmic.
8. Keep estimator, back end, filtering, environment dynamics, floor texture, domain, and hardware separate unless their interaction is the declared factor.
9. Record every switch that changes estimator behaviour in the run manifest, not only those the launch file exposes. Loop-closing and global-bundle-adjustment state are currently absent from `run_metadata.csv`, so two ORB-SLAM3 runs of the same dataset in opposite configurations produce manifests that differ only in identifier and timestamp.
10. Repeat every configuration of a nondeterministic estimator at least three times and report median and spread. ORB-SLAM3 is multi-threaded and its run-to-run variation on identical input is presently unmeasured, which places a floor on the differences that can be attributed to any factor.

Material deviations from the originally proposed method are given in [table 3.13](#). They are differences between the proposal and the system that exists, not failures: each is recorded so that no proposal capability is read into the implemented pipeline.

These deviations define the actual system at the reporting cutoff. Chapter 4 must report only measurements satisfying the validity rules above, while failed, excluded, and retracted runs remain in a clearly labelled diagnostic record.

Repository Evidence Used for This Draft

The principal evidence comprises `README.md`; the four `MEMORY_CLAUDE/README.md` engineering records; the ORB-SLAM3 integration record; `docs/report/project_limitations_and_map_benefits.md`; the source, launch, and configuration files cited in this chapter; and `script/vio_metrics.py`, `script/extract_gt.py`, `script/yolo_eval.py`, `script/fit_map_pose.py`, and `script/validate_map.py`. These project artifacts

Table 3.13: Planned-versus-implemented deviations at the reporting cutoff. “Added” marks work not present in the original proposal.

Component	Proposed	Implemented	Status
Semantic segmentation	pixel masks, as in the reference RY-SLAM approach	three-class bounding boxes, no motion gate	deviation
Dynamic rejection point	not specified	ORB-SLAM3 rejects before octree distribution; VINS-Fusion also removes persistent KLT tracks	implemented
Estimator coupling	shared output representation	two independent estimators, not one fused estimator	deviation
RTAB-Map correction	integrated back end	separate process consuming external TF odometry; correction never injected into estimator state	deviation
Real-robot evaluation	metric accuracy against ground truth	calibration-limited, no independent trajectory truth	blocked
Metric infrastructure	resource, throughput, uncertainty, manifests	partially implemented; RPE now complete	incomplete
Relocalization and merging	robust arbitrary-start localization, aliasing rejection, extend-mode validation, front-end global optimization, multi-session merging	partially verified relocalization only	planned
Non-visual baseline	not proposed	wheel, IMU and EKF estimators with a data-derived noise model (section 3.6)	added
Covariance consistency	not proposed	NEES and σ -coverage testing for the non-visual estimators	added

establish local implementation. They do not replace primary literature citations for the scientific origin of the algorithms.

Chapter 4

Experiments and Research Results

Reporting cutoff: 16 September 2026.

Purpose: Advisor progress update, not a final performance claim.

Analysis status: Completed for the relogged artifacts. Root-cause debugging was intentionally deferred.

4.1 Relogged Campaign and Validity Audit

The relogged campaign contains 66 estimator runs across seven featureless-floor simulation datasets. Each condition was requested three times. The four dynamic datasets contain baseline and nominal YOLO-filtered folders for both estimators ($4 \times 3 \times 4 = 48$ runs). The three static datasets contain baseline ORB-SLAM3 and VINS-Fusion runs ($3 \times 3 \times 2 = 18$ runs). All directories contain a trajectory, metadata, process-performance records, module records, and a graceful-shutdown summary.

The validity audit distinguishes an operational estimator run from a valid treatment comparison. All 66 runs initialized and produced usable log rows. Two ORB-SLAM3 runs—`dataset_static_nofloortexture_000/logging_20260916_02` and `logging_20260916_03`—have incomplete `run_summary.csv` schemas: their final inertial and map counters are absent. Their frame-level `performance.csv` records nevertheless reach `imu_initialized=1`, `inertial_ba1=1`, and `inertial_ba2=1`, while `local_mapping.csv` records mapping activity. They are therefore retained as operational runs; the missing summary fields are reported as a logging anomaly rather than interpreted as estimator failure.

Ground truth was extracted directly from each exact-name dataset rosbag’s `/ground_truth/odometry` topic and associated with every repeat using source header timestamps. Dataset association follows the containing folder name, not the path recorded in `run_metadata.csv`. The PC copies were used for `dataset_dynamic_nofloortexture_01_000` and `dataset_static_nofloortexture_001`; the other five bags were read from the removable drive. That split was not a preference: both drive copies are unreadable despite carrying complete metadata, as recorded in [table 3.9](#). Each estimate is aligned to truth by a rigid $SE(3)$ fit with scale fixed. Translational ATE RMSE and rotational geodesic RMSE can therefore be reported. Relative pose error was added to the analysis module after the cutoff and is reported in [section 4.5.1](#); it required no estimator re-runs.

The nominal VINS+YOLO treatment did not pass the mechanism check. Although its metadata says `filter=true`, all 12 runs report zero matched detection frames, zero nonzero mask coverage, and zero removed dynamic tracks. These runs remain useful for operational and compute inspection but cannot test semantic filtering. ORB+YOLO

Table 4.1: Relogged experiment progress and validity. “Operational” requires graceful shutdown, estimator initialization, at least ten processed frames, and a trajectory. Treatment validity additionally requires the intended filter to be active and paired input counts and durations to differ by no more than 5%.

Run family	Attempted	Complete	Operational	Valid pairs	Interpretation
Static baseline	18	18	18		– two incomplete ORB summaries
Dynamic baseline	24	24	24		– descriptive estimator logs
Dynamic ORB+YOLO	12	12	12	1 of 12	11 input-mismatched pairs
Dynamic VINS+YOLO	12	12	12	0 of 12	filter inactive in every run
Total	66	66	66	1 treatment pair	preliminary campaign

recorded 9,706 matched and 10,764 missed detector frames over 20,470 mask-use rows, with no rejected masks. Its median mask coverage was zero (IQR 0.0798), reflecting many frames without a used mask. Eleven of its 12 baseline/filter pairs differed in received image count by more than 5%; only one pair satisfies the predefined pairing rule. Consequently, no cross-treatment accuracy conclusion is made.

4.2 Non-Visual Baseline and Estimator Comparison

4.2.1 Question, hypotheses, and conditions

Until this experiment, every trajectory figure in this chapter reported visual error against ground truth with nothing to compare it to, so a reader could see how large the error was but not whether it was good. This section establishes the dead-reckoning floor. The question is whether a non-visual estimator—wheel odometry, IMU dead reckoning, or an EKF fusing them—provides usable localization on the same recorded data, and whether the uncertainty each one reports describes the error it actually makes.

Four hypotheses were registered before the metrics were computed:

- H1 Wheel-odometry drift is dominated by the measured yaw-rate scale error, so the two bags, whose scales err +1.1% and −4.7%, should bow in opposite rotational directions.
- H2 IMU-only dead reckoning diverges by orders of magnitude more than wheel odometry over the same interval.
- H3 The EKF improves on both single-sensor estimators.
- H4 All three report overconfident covariances, because a dead-reckoning noise model omits systematic scale error.

The independent variable is the estimator; everything else is held fixed within a dataset. Table 4.2 records the controlled conditions. All five estimators consume the same recordings and all visual runs replayed them under identical settings.

The five datasets are not repeats of one condition. Only `allsensor_000` and `allsensor_001` share a world. The remaining three vary two factors: object placement, where `objonwallonly` restricts obstacles to the wall region and clears the central floor, and floor

texture, where **bayline** adds painted bay lines to a surface that is otherwise featureless. No quantity is pooled across worlds; each dataset is read as its own condition.

Table 4.2: Controlled conditions for the estimator comparison. All five estimators consume the same two recordings and are scored against the same ground truth over the same interval.

Factor	Level
Worlds	four: <code>..._static_01_nofloortexture</code> (000, 001), <code>..._static_nofloortexture_objonwallonly</code> (002), <code>..._static_bayline_objonwallonly</code> (003), <code>..._static_01_bayline</code> (004)
Floor	plain (000–002) versus bayline, i.e. painted lines (003, 004)
Scene dynamics	static
Floor	featureless
Datasets	five, 81.8 s to 102.4 s of simulated time and 113.1 m to 142.3 m driven each
Dynamic filter	disabled in every run (<code>filter=false</code>)
Replay rate	0.5× for both visual estimators
Ground truth	<code>/ground_truth/odometry</code> , 50 Hz, header stamps
Alignment	rigid $SE(3)$, scale fixed, for cross-system comparison
Experimental unit, n	one bag; $n = 5$ across <i>four</i> worlds. Only <code>static_01</code> plain is repeated, and only twice
Visual configurations	VINS-Fusion; ORB-SLAM3 with loop closing enabled and disabled, one run each

4.2.2 Validity

All ten runs passed their predefined checks. The three baselines completed on both bags with the planar assumption holding by a wide margin (maximum tilt 0.0018° against a 1° threshold), the bicycle yaw-rate model selected automatically on both, `imu_orientation_used=NO` recorded in every run, and the EKF applying 4355 and 4091 wheel corrections, matching the wheel-odometry message counts exactly. Both VINS-Fusion runs processed every image received (2639 and 2478), initialized once in 0.33 s, and recorded no resets. Both ORB-SLAM3 runs shut down gracefully with no tracking-loss events and inertial initialization at 2.48 s and 2.51 s.

[Queried] ORB-SLAM3 receives fewer stereo pairs than are available on every dataset, while VINS-Fusion receives all of them (2639, 2478 of 2479, 2628, 3102 and 3046). The ORB shortfall is 3.8 % to 11.9 % and varies by run as well as by dataset: 90.2 % and 94.2 % of the available pairs on `allsensor_000` for the two configurations, 96.2 % and 95.1 % on `allsensor_003`. Its own counters report zero decode failures, and the stereo timestamps are aligned to 0 s median offset with 100 % of pairs inside the 20 ms synchronisation slop, so synchronisation is not what loses them.

The counter itself cannot say where they go: `stereo_pairs_received` is incremented inside the *synchronised* callback, and the node keeps no count of images arriving. What the source does show is an asymmetry in input buffering. ORB-SLAM3 subscribes with `rclcpp::SensorDataQoS()`, which is best-effort with a depth of five, so a frame arriving while the tracking callback is busy is discarded by the middleware and never recorded anywhere; VINS-Fusion subscribes with `KeepLast(100).reliable()` and buffers the same

burst. The shortfall is therefore most consistent with ORB-SLAM3 dropping frames at its own subscriber under load, not with loss in transport. The loop-closure-free runs support that reading on three datasets, receiving more pairs than the enabled runs on `allsensor_000`, `_001` and `_002` while doing less work per frame, though `allsensor_003` runs the other way and the mechanism is not proven. Trajectory accuracy over a shared interval is not obviously affected—ORB-SLAM3 is the more accurate of the two on some datasets despite the shortfall—but no throughput, latency or per-frame compute comparison between the two estimators is admissible until the loss is explained: the two estimators differ not only in load but in input-queue policy, so such a comparison would measure the QoS choice as much as the algorithm.

[Ruled out] A plausible reading of these results is that the visual estimators lose to the EKF because their input rate collapsed. The data does not support it. Source imagery is regular at 30.3 Hz in simulated time on all five bags, with a median inter-frame interval of 0.0330 s and no gap exceeding 0.1 s; VINS-Fusion received and processed every frame on every dataset, with zero skipped before the back end, and still trails the cross-validated EKF on four of five. Frame loss is real for ORB-SLAM3 and remains unexplained, but it cannot account for the margin, because the estimator with no frame loss at all shows the same one.

[Queried] ORB-SLAM3 was run in two configurations on every dataset, recorded as `logging_20260916_01` with loop closing enabled and `logging_20260916_02` with it disabled, and both are reported below as separate systems. Within the enabled configuration the loop closer behaves very differently per dataset (table 4.3), so those five trajectories are not repeats of one condition and must not be pooled.

A counter reading `loop_closures,0` does *not* mean the loop closer was inactive. On `allsensor_000` it performed 447 place-recognition checks, found 90 loop candidates and rejected all 90 on inertial geometry; on `allsensor_004` it found 114 and accepted none. The disabled runs are distinguishable only in `loop_closing.csv`, where every event carries `current_keyframe_id = -1` and a duration below 0.003 ms, the signature of a thread spinning with nothing queued to it.

Table 4.3: Loop-closing activity in the enabled ORB-SLAM3 configuration (`logging_20260916_01`). Accepted closures are a small and dataset-dependent fraction of candidates, and two datasets accept none at all despite the closer running throughout. In the disabled configuration (`logging_20260916_02`) no keyframe ever reaches the loop closer on any dataset.

Dataset	Place-recognition events	Candidates	Accepted	Global BA
<code>allsensor_000</code>	537	90	0	0
<code>allsensor_001</code>	379	2	2	2
<code>allsensor_002</code>	274	11	1	1
<code>allsensor_003</code>	591	64	1	0
<code>allsensor_004</code>	713	114	0	0

[Provenance gap] `run_metadata.csv` records 22 parameters and loop-closure state is not among them, so the whole distinction rests on the directory name—and that name does not mean the same thing everywhere in this chapter. In the relogged campaign of section 4.1, `logging_20260916_01` through `_03` are three *repeats* of one configuration, all with an active loop closer. In the `allsensor` campaign reported here, `_01` and `_`

02 are two *configurations*, loop closing enabled and disabled, with one run each. The same suffix therefore encodes a repeat index in one section and an experimental factor in the next. The metadata for the two bag-000 runs is identical apart from the experiment identifier, output path and start time, and both name the same installed configuration file, so nothing in the recorded provenance distinguishes an enabled run from a disabled one. The distinction used in this chapter was recovered from `loop_closing.csv` after the fact. The logger should record the loop-closing and global-BA switches explicitly before any further ORB-SLAM3 campaign.

4.2.3 Accuracy results

Table 4.4 reports all five estimators. Aligned ATE is the only column comparable across systems, because the visual estimators build their own world frames while the baselines start from the ground-truth pose.

Table 4.4: Estimator comparison on the five `allsensor` bags, which span four worlds and both floor conditions; they are not repeats of one another except for `allsensor_000` and `allsensor_001`. Bold marks the best aligned ATE of each dataset—the EKF on four of five, VINS-Fusion on `allsensor_004`. “Aligned ATE” is RMSE after a rigid $SE(3)$ fit with scale fixed, the convention used throughout this chapter. “Anchored” columns are the direct difference from ground truth with no alignment and are defined only for the estimators that start from the ground-truth pose; the visual estimators’ own world frames make that subtraction meaningless, so those cells are set to `–`. Drift is the final position error as a fraction of distance driven.

Dataset	World	Estimator	Aligned ATE [m]	Anchored RMS [m]	Anchored final [m]	Drift [%]	Final yaw [°]
allsensor_000	static_01 plain	IMU dead reckoning	21.697	39.353	47.597	40.66	+2.19
		Wheel odometry	2.801	4.732	4.767	4.07	–13.76
		VINS-Fusion	0.616	–	–	–	–
		ORB-SLAM3	0.579	–	–	–	–
		ORB-SLAM3 (no LC)	0.285	–	–	–	–
		EKF wheel+IMU	0.190	0.281	0.122	0.10	+0.19
allsensor_001	static_01 plain	IMU dead reckoning	12.617	14.728	23.741	20.99	+1.33
		Wheel odometry	0.979	1.826	2.426	2.15	–12.95
		VINS-Fusion	0.383	–	–	–	–
		ORB-SLAM3	0.372	–	–	–	–
		ORB-SLAM3 (no LC)	0.516	–	–	–	–
		EKF wheel+IMU	0.308	0.351	0.303	0.27	+0.11
allsensor_002	objonwallonly plain	IMU dead reckoning	13.237	16.359	17.728	15.39	+1.36
		Wheel odometry	1.485	2.888	4.332	3.76	–30.11
		VINS-Fusion	0.558	–	–	–	–
		ORB-SLAM3	0.602	–	–	–	–
		ORB-SLAM3 (no LC)	1.796	–	–	–	–
		EKF wheel+IMU	0.256	0.345	0.346	0.30	–0.68
allsensor_003	bayline_ objonwallonly bayline	IMU dead reckoning	45.562	62.903	74.745	53.16	–5.84
		Wheel odometry	3.192	3.979	9.231	6.57	–37.17
		VINS-Fusion	0.784	–	–	–	–
		ORB-SLAM3	6.481	–	–	–	–
		ORB-SLAM3 (no LC)	0.515	–	–	–	–
		EKF wheel+IMU	0.423	0.668	1.240	0.88	–7.20
allsensor_004	static_01_ bayline bayline	IMU dead reckoning	26.709	41.476	48.099	33.81	–1.35
		Wheel odometry	3.241	5.124	5.791	4.07	–17.68
		VINS-Fusion	0.425	–	–	–	–
		ORB-SLAM3	1.237	–	–	–	–
		ORB-SLAM3 (no LC)	0.905	–	–	–	–
		EKF wheel+IMU	0.494	0.776	0.831	0.58	–5.30

H2 and H3 are supported on every dataset. IMU-only dead reckoning is 7.7 to 14.3 times worse than wheel odometry, and the EKF improves on both single sensors by an order of magnitude everywhere.

H3’s stronger form—that the EKF also beats the visual estimators—holds on four of the five datasets but not universally, and the margin is much narrower than the three-dataset version of this section reported. **That earlier figure of “3.0, 1.2 and 2.2 times” was computed against ORB-SLAM3 with loop closing enabled, which is not the best visual result available.** Measured against the best visual estimator of each dataset, and using the cross-validated EKF so that no in-sample tuning advantage is carried (table 4.8), the margins are 1.38, 1.41, 2.53, 1.31 and 1.13. On `allsensor_004` the in-sample EKF at 0.494 m is beaten outright by VINS-Fusion at 0.425 m; only the cross-validated figure of 0.377 m is ahead. The honest statement is that a five-state proprioceptive filter is competitive with, and usually better than, stereo visual-inertial SLAM in these scenes—not that it dominates it.

Two expectations about scene content are not supported. Clearing the central floor did not penalise vision: the visual errors on `allsensor_002` (0.558 and 0.602 m) sit close to `allsensor_000` (0.616 and 0.579 m). **Nor did adding the bay markings help it**—the two `bayline` datasets give VINS-Fusion 0.784 and 0.425 m against 0.616, 0.383 and 0.558 m on the plain floors, which is no improvement. That null result is weaker evidence than the factor’s name implies: both floor conditions rest on the same flat, untextured base and differ only by fourteen painted bay outlines covering a small fraction of the floor (figure 3.5), so what was varied is bay markings, not surface texture. What the textured datasets did change is the *proprioceptive* side: both carry the largest wheel yaw-rate scale errors of the five, and both degrade the EKF accordingly (section 4.2.4).

The mechanism is visible in the last two columns and is the substantive finding of this section: *the two proprioceptive sensors fail in complementary ways*. The IMU holds heading to 2.19° and 1.33° while losing position catastrophically; the wheels hold position to a few percent while losing 13° of heading. The EKF is better than either in *both* quantities simultaneously, reaching 0.19° of final heading error—an order of magnitude better than the gyroscope alone—because the wheel yaw rate is used to observe the gyroscope bias rather than heading, so the filter never inherits the wheels’ own heading drift (section 3.6).

4.2.4 Falsification of H1, and the drift mechanism

H1 is **falsified** across all five datasets. If drift followed the fitted yaw-rate scale, the bag with the most nearly correct scale would drift least. The opposite holds (table 4.5): `allsensor_002` has the best scale of the five, +0.34%, and −30.11° of heading error, while `allsensor_004` has the worst scale, −7.50%, and only −17.68°. All five heading errors share a sign; the scale errors do not.

The proximate mechanism is nevertheless now identified, and it is simpler than a scale error. The wheel model’s yaw rate carries a small persistent *offset* against ground truth, averaged over all moving samples: −0.002 77, −0.002 85 and −0.006 24 rad s^{−1} on the three bags. Because heading is the time integral of that rate, an offset produces an error that grows linearly with elapsed time rather than with rotation. Multiplying each offset by its run duration reproduces the measured error to within 1.4° on all five datasets (table 4.5, last column), including the two textured-floor runs that were recorded after the mechanism was proposed. The drift is accounted for.

[Corrected] An earlier reading of this experiment proposed that the offset came from the bicycle model averaging the two kingpin angles, since tan of a mean is not the mean

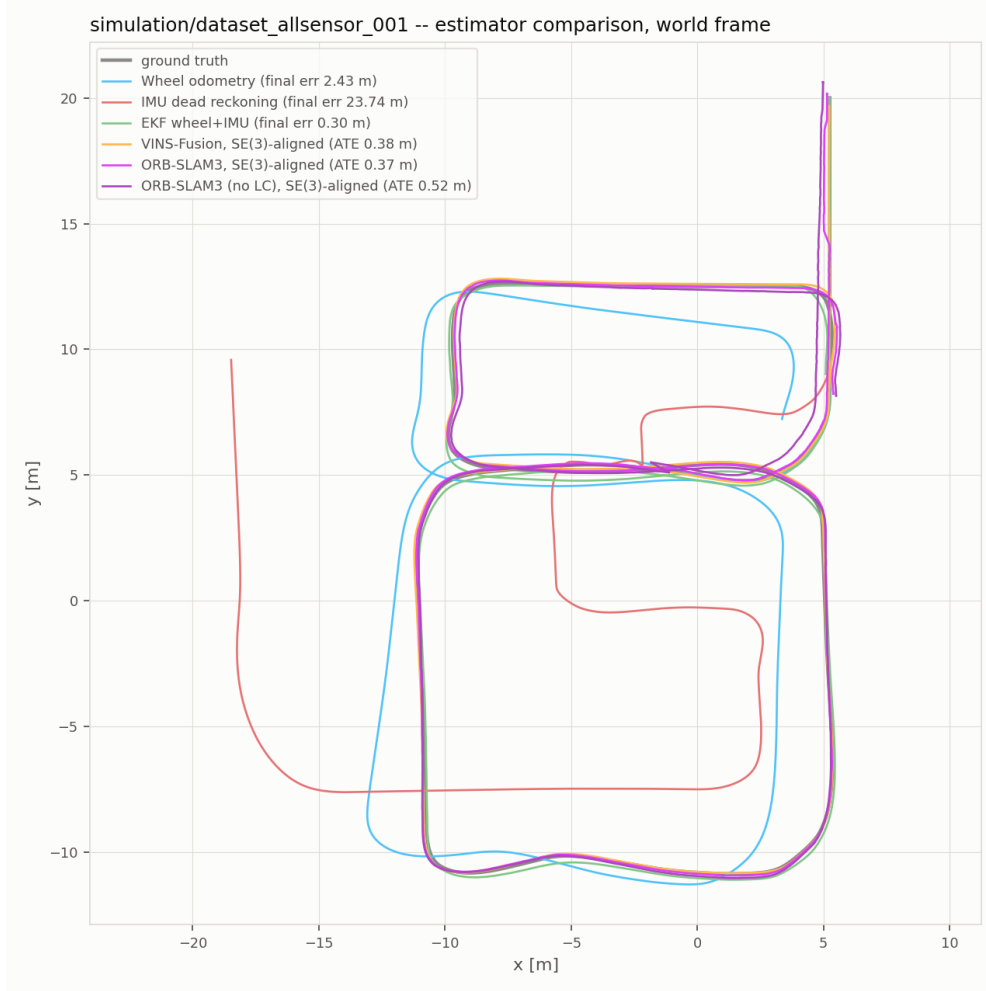


Figure 4.1: Estimator comparison on `dataset_allsensor_001`, world frame. The baselines are drawn as estimated; the visual estimators are drawn after the same rigid $SE(3)$ alignment used for their metrics, because their own world frames are arbitrary. Ground truth is hidden beneath the EKF, VINS and ORB traces for most of the run. Wheel odometry (blue) bows consistently outward; IMU dead reckoning (red) leaves the building.

Table 4.5: Wheel-odometry heading drift against the fitted yaw-rate scale. “Rotation” is the total unsigned ground-truth rotation of the run, and “error/rotation” expresses the final heading error as a fraction of it. Scale error and heading error correspond in neither sign nor magnitude. The last column is the constant-offset prediction described below: the mean yaw-rate residual of the run multiplied by its duration, which matches the measured error to within 1.4° on every dataset, and to within 0.5° on three of five.

Dataset	Yaw-rate scale [%]	Final yaw error [$^\circ$]	Rotation [$^\circ$]	Error/rotation [%]	Offset prediction [$^\circ$]
<code>allsensor_000</code>	+1.10	-13.76	536	-2.566	-13.81
<code>allsensor_001</code>	-4.73	-12.95	852	-1.519	-13.37
<code>allsensor_002</code>	+0.34	-30.11	865	-3.481	-31.03
<code>allsensor_003</code>	-6.25	-37.17	1123	-3.309	-38.51
<code>allsensor_004</code>	-7.50	-17.68	895	-1.976	-18.03

of tan. That explanation is *wrong* and is withdrawn. Recomputing the yaw rate as the mean of each wheel’s own bicycle-equivalent rate changes the residual from -0.00277 to -0.00288 , from -0.00285 to -0.00296 , and from -0.00624 to $-0.00630 \text{ rad s}^{-1}$ — under 4 % in every case, two orders of magnitude short of what the observed drift requires. The averaging is not the source.

What remains open is the origin of the offset itself. It is not a pure magnitude error: splitting the residual by turn direction gives $+0.0109$ against -0.0142 on `allsensor_000` and $+0.0075$ against $-0.0292 \text{ rad s}^{-1}$ on `allsensor_002`, so the model under-rotates in both directions *and* carries a directional bias on top. On `allsensor_001`, whose route returns to its starting heading, both directions are negative and only the bias is visible. A steering-angle zero offset, an unmodelled slip angle, and a kingpin-geometry error are all consistent with what has been measured so far and none has been tested.

This matters beyond wheel odometry, and the two textured-floor datasets turn what was a caveat into a measurement. The EKF observes its gyroscope bias through exactly this wheel yaw rate (section 3.6), so a persistent offset in the wheel rate is absorbed into the estimated bias and reappears as heading error. Table 4.6 shows the two tracking each other across the five datasets: where the wheel yaw-rate scale is within 1.1 % of unity the EKF holds heading to 0.42° or better, and where it is 6 to 7.5 % low the EKF’s heading error rises roughly tenfold.

Table 4.6: The EKF’s heading error follows the quality of the wheel yaw rate it uses to observe its gyroscope bias. The filter is no better than its rate reference, which is the cost of making the bias observable at all.

Dataset	Floor	Wheel yaw scale	Wheel yaw σ [rad s ⁻¹]	EKF yaw RMS [°]	EKF ATE [m]
<code>allsensor_000</code>	plain	1.0110	0.0659	0.35	0.190
<code>allsensor_002</code>	plain	1.0034	0.0705	0.30	0.256
<code>allsensor_001</code>	plain	0.9527	0.0804	0.42	0.308
<code>allsensor_003</code>	bayline	0.9375	0.1192	3.54	0.423
<code>allsensor_004</code>	bayline	0.9250	0.0613	3.41	0.494

An earlier version of this section, written when only the three plain-floor datasets existed, recorded that the EKF held heading below 0.7° everywhere and inferred that the absorbed error was small. *That inference does not survive the textured-floor datasets* and is withdrawn: the coupling is the dominant term in the EKF’s heading error whenever the wheel rate is poor, and it sets a floor on the filter’s accuracy that no amount of gyroscope quality can lift.

One prediction of the original reasoning did survive, and is visible in figure 4.2: wheel-odometry error stays flat at roughly 6 mm for the first 9.5 s, the initial straight run, and only grows once the first turn begins. Turning, not distance, starts the error; the offset then sustains it.

4.2.5 Covariance consistency

H4 is supported emphatically. Table 4.7 reports the average normalised estimation error squared, $\text{NEES} = \mathbf{e}^\top \mathbf{P}^{-1} \mathbf{e}$ with $\mathbf{e} = [\Delta x, \Delta y, \Delta \theta]$, which a correctly scaled covariance makes average 3.0, together with the fraction of the run for which the truth lies inside the reported $k\sigma$ bounds.

Table 4.7: Covariance consistency of the non-visual estimators. A consistent filter gives ANEES = 3.0 and σ -coverages of 0.683, 0.954 and 0.997. Every estimator is overconfident by roughly two orders of magnitude. Neither VINS-Fusion nor ORB-SLAM3 writes a pose covariance, so neither can be tested here.

Dataset	Estimator	ANEES	1σ	2σ	3σ	Verdict
allsensor_000	Wheel odometry	569.4	0.309	0.390	0.408	overconfident
	IMU dead reckoning	311.0	0.005	0.022	0.043	overconfident
	EKF wheel+IMU	64.7	0.426	0.767	0.879	overconfident
allsensor_001	Wheel odometry	263.9	0.015	0.131	0.187	overconfident
	IMU dead reckoning	339.9	0.001	0.009	0.021	overconfident
	EKF wheel+IMU	123.2	0.194	0.439	0.537	overconfident
allsensor_002	Wheel odometry	729.4	0.140	0.143	0.145	overconfident
	IMU dead reckoning	244.7	0.017	0.045	0.147	overconfident
	EKF wheel+IMU	114.5	0.193	0.350	0.501	overconfident
allsensor_003	Wheel odometry	406.7	0.260	0.343	0.459	overconfident
	IMU dead reckoning	1504.3	0.010	0.018	0.035	overconfident
	EKF wheel+IMU	79.1	0.034	0.291	0.476	overconfident
allsensor_004	Wheel odometry	785.0	0.154	0.157	0.162	overconfident
	IMU dead reckoning	289.5	0.004	0.017	0.044	overconfident
	EKF wheel+IMU	78.6	0.254	0.330	0.570	overconfident
<i>consistent reference</i>		3.0	0.683	0.954	0.997	–

The IMU case is the starkest: its 3σ envelope contains the truth for between 2.1% and 14.7% of each run, and its worst ANEES, 1504 on **allsensor_003**, is the largest figure in the experiment. A consumer weighting this estimate by its stated covariance—a costmap, a pose-graph edge, a relocaliser gate—would trust it tens of times more than it deserves. Wheel odometry on **allsensor_002** is the single worst case in the experiment, ANEES 785 on **allsensor_004** with 3σ coverage of 0.162, which is consistent with the heading offset of [section 4.2.4](#): the covariance model contains no term for a persistent rate bias, so the larger that bias, the further the claimed uncertainty falls behind the real error. The EKF is the least bad on every dataset, at 3σ coverage between 0.476 and 0.879, and [figure 4.2](#) shows why: its NEES is genuinely near 3 during the opening straight and only departs once turning begins, which is the same moment the unmodelled heading error appears.

[Method caveat] The χ^2 interval behind the verdict column assumes independent samples. At 200 Hz consecutive dead-reckoning errors are almost perfectly correlated, so the test is run on a 1 Hz subsample (81 to 102 samples per dataset). That reduces the dependence but does not remove it, and the verdicts should be read as indicative. The σ -coverage columns make no independence assumption and are the sturdier statement.

4.2.6 Cross-validation of the EKF noise model

The EKF’s noise model is derived from the same recording it runs on, whereas the visual estimators receive no comparable tuning. The wheel and IMU baselines are unaffected—their measured σ values enter only the covariance, never the trajectory—but the EKF’s

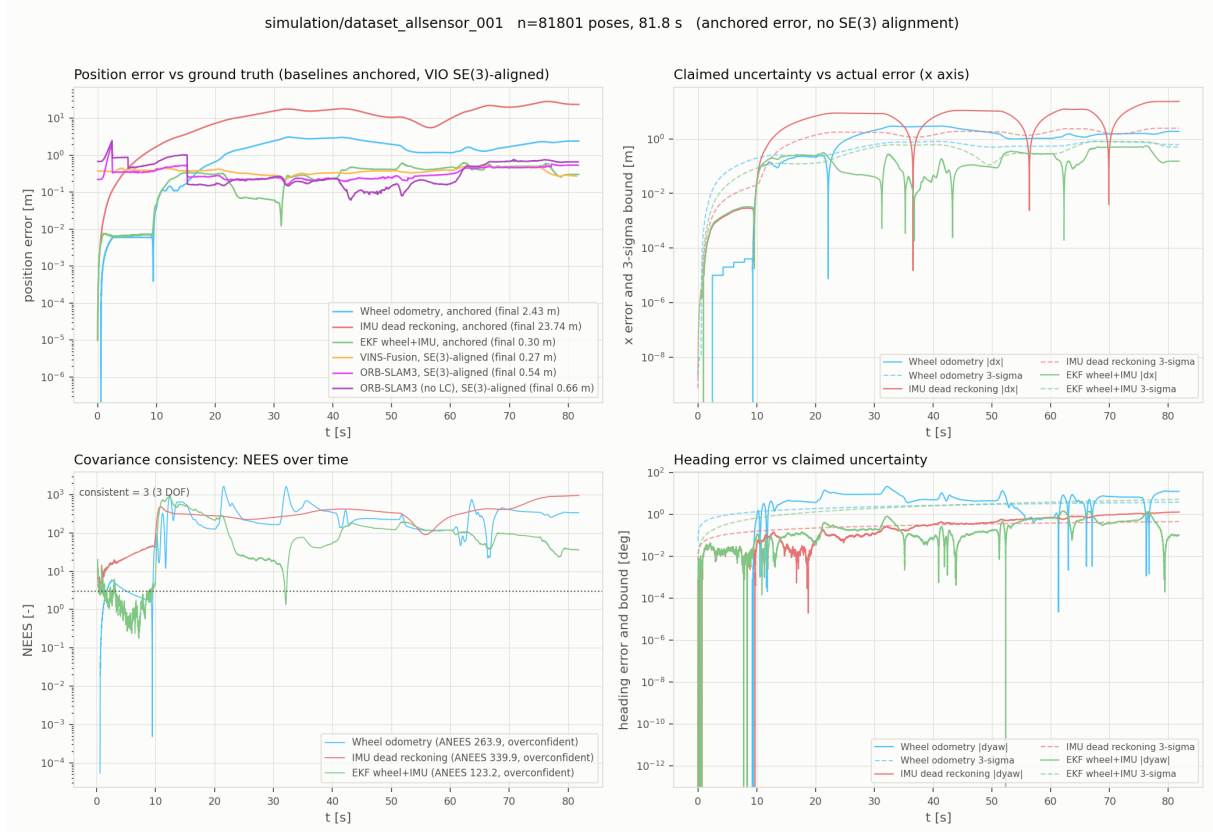


Figure 4.2: Error against claimed uncertainty for dataset_allsensor_001. Top left: position error over time, baselines anchored and visual estimators $SE(3)$ -aligned. Top right and bottom right: absolute error against the estimator's own 3σ bound, in x and in heading; where the solid line sits above the dashed line of the same colour the filter is overconfident. Bottom left: NEES against the consistent value of 3. The step near 9.5 s in every panel is the first turn of the run.

values enter the Kalman gain and so do change its output. The comparison was therefore repeated with the noise models exchanged between bags (table 4.8).

Table 4.8: EKF accuracy with an in-sample and an out-of-sample noise model. The change is negative on four of five bags—the borrowed model is *better* there—and the mean change is about -10% , which is the signature of no systematic in-sample advantage.

Dataset	Noise model from	Aligned ATE [m]	Change
allsensor_000	same bag	0.190	–
	allsensor_001	0.206	+8.8%
allsensor_001	same bag	0.308	–
	allsensor_000	0.263	–14.5%
allsensor_002	same bag	0.256	–
	allsensor_000	0.220	–13.9%
allsensor_003	same bag	0.423	–
	allsensor_000	0.392	–7.1%
allsensor_004	same bag	0.494	–
	allsensor_000	0.377	–23.8%

The noise model transfers between runs, and on four of the five bags the borrowed model is the better one, which no in-sample advantage could produce. The EKF’s accuracy is therefore a property of the estimator rather than of having seen the answer. Cross-validated it reaches 0.206, 0.263, 0.220, 0.392 and 0.377 m, which is 1.38, 1.41, 2.53, 1.31 and 1.13 times better than the best visual estimator of each dataset. Notably the cross-validated figure on `allsensor_004` recovers a lead that the in-sample filter did not have.

4.2.7 Interpretation and limitations

Within these two recordings, a five-state planar filter over wheel encoders and a gyroscope localises more accurately than either stereo visual-inertial system. That is an uncomfortable result for the thesis premise and is reported as measured. It does not show that vision is unnecessary; it shows that in this condition vision has not yet earned its compute, and that the justification has to come from capabilities dead reckoning does not have at all—loop closure, relocalization after tracking loss, and the map itself—or from conditions these bags do not contain.

[Generalisation limit] The comparison is confined to *static* scenes. Four worlds and both floor conditions are now covered and the ranking is stable across them, so the earlier form of this limitation—that the result held only on featureless floors, vision’s worst case—is no longer the binding one. Adding the bay markings did not improve either visual estimator, which removes that explanation rather than confirming it—though the markings are a much weaker manipulation than “textured floor” suggests (figure 3.5), so a genuinely textured surface remains untested.

What remains is that every dataset carrying wheel encoders is static (table 3.9). The result must not be restated as “proprioception outperforms visual SLAM” without that qualifier, and it cannot be extended to dynamic scenes—where wheel odometry is unaffected by moving objects, and where the visual systems’ known failures occur—until

the `allsensor` topics are recorded in a dynamic world. That recording remains the single most informative next run in this line of work.

[Queried] Both visual estimators replayed their bags at $0.5\times$ real time, so each frame received roughly twice the wall-clock budget it would have online. The accuracy figures in [table 4.4](#) are therefore not evidence of real-time capability, and no timing conclusion may be drawn from this experiment. The three baselines, being offline, have no replay rate at all, which is a second asymmetry in the same comparison.

[Statistical limit] $n = 5$ across four worlds, and no condition is repeated more than twice. The tables therefore report individual values deliberately; no mean, standard deviation or confidence interval is computed, and between-dataset differences are not evidence of a systematic effect. A single run per world cannot establish a scene-content effect; it shows only that the estimator ranking survives changes of scene.

[Unmeasured variance] ORB-SLAM3 is multi-threaded and not deterministic, and its run-to-run spread on identical input has not been measured here: each of its two configurations was run once per dataset. Several differences this chapter reports sit within a range that spread could plausibly cover—most sharply 6.481 m with loop closing enabled on `allsensor_003` against 0.515 m with it disabled, and the reversal on `allsensor_002` where disabling it is three times *worse*. Those differences are real measurements of the runs that were made, but they cannot yet be attributed to the loop-closing switch. Three or more repeats per configuration, reported as median and spread in the manner of the relogged campaign ([section 4.1](#)), are required before any loop-closure conclusion is drawn.

Two further limitations are structural rather than queried. All three baselines begin from the ground-truth pose, because dead reckoning has no global reference; a deployed system would have to obtain that pose some other way, which is exactly the relocalization problem listed as future work. And the covariance result cannot yet be extended to the visual estimators, because neither writes a pose covariance—adding that to VINS-Fusion’s marginalisation output is an instrumentation change, not an analysis one, and is the prerequisite for applying this consistency test to the pipelines the thesis is actually about.

4.3 Offline YOLO Detector Performance

YOLO was evaluated independently of either SLAM integration on all four dynamic no-floor-texture rosbags. Ground-truth object poses came from `/world/default/pose/info`; because that topic’s message headers are zero, its rosbag receive times were mapped to simulation time by interpolation against the recorded `/clock` stream. Known model meshes were transformed into the camera frame and projected into `cam0` to form two-dimensional boxes. Sampled overlays from every bag were inspected before calculating metrics and showed spatially plausible agreement between the projected boxes and visible objects.

4.3.1 Bounding-Box Detection and Localization

The evaluation used 9,860 camera frames, confidence threshold 0.35, NMS IoU threshold 0.50, and matching IoU threshold 0.50. Ground-truth boxes smaller than 12 pixels on either side were omitted; heavily occluded ground-truth boxes and predictions explained by explicitly unlabelled scenery were ignored rather than counted as errors. [Table 4.9](#)

reports per-dataset results and a micro-average formed by pooling TP, FP, and FN counts across all frames.

Table 4.9: Offline YOLO performance against projected Gazebo object ground truth. This is a detector-capability measurement, not an evaluation of delivery to the online SLAM filters.

Dataset	Frames	TP	FP	FN	Precision	Recall	F1	Moving recall
Dynamic 00/000	2,347	9,987	4,826	5,900	0.674	0.629	0.651	0.617
Dynamic 00/001	2,236	10,158	4,265	6,868	0.704	0.597	0.646	0.599
Dynamic 01/000	2,564	8,142	4,226	6,064	0.658	0.573	0.613	0.568
Dynamic 02/000	2,713	8,661	3,743	6,502	0.698	0.571	0.628	0.581
Pooled	9,860	36,948	17,060	25,334	0.684	0.593	0.635	0.591

The pooled evaluation contains 62,282 scored ground-truth boxes, of which 46,491 are moving, plus 34,476 projected boxes excluded as heavily occluded. The model produced 58,776 raw predictions. Mean image coverage was 10.42% for all scored ground truth, 7.88% for moving ground truth, and 9.07% for predictions. At the stated thresholds, the detector misses approximately 41% of scored moving objects; moving-object recall ranges from 0.568 to 0.617 between datasets. Conversely, the results demonstrate that the trained model can produce detections on every recorded bag. They do not contradict the zero-box VINS+YOLO logging anomaly: offline detector capability and online message delivery/timestamp matching are different mechanisms. Average precision is not reported because this evaluation retained predictions only above a single confidence threshold.

YOLO evaluation using /clock-synchronized Gazebo world poses

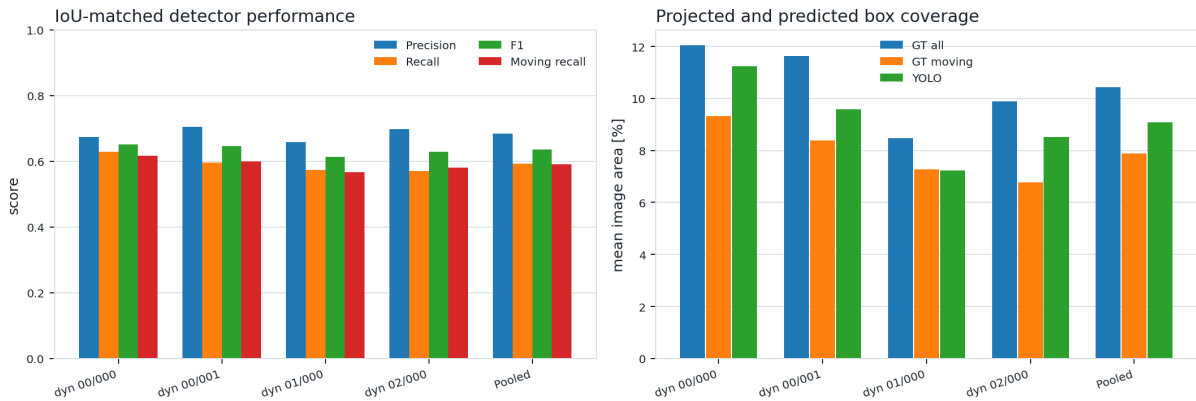


Figure 4.3: Per-dataset and pooled detector scores and box-area coverage. Pooled precision, recall, F1, and moving-object recall are calculated from pooled counts, rather than by averaging the four displayed dataset scores.



Figure 4.4: Representative validation overlay used before accepting the offline metrics. Green boxes are projected ground truth, red boxes are YOLO predictions, and yellow boxes are occluded or ignored regions. An asterisk marks a ground-truth object classified as moving from its world-pose history.

4.3.2 Object-Class Assignment Conditional on Localization

This experiment is not whole-image classification: an image can contain many objects, and the model outputs a class for each predicted bounding box. To keep class assignment separate from localization, predictions and ground-truth boxes were greedily associated by $\text{IoU} \geq 0.50$ without requiring their classes to agree. Classification was then evaluated only on the 37,061 localized object pairs. Missed ground-truth objects and unmatched predictions remain bounding-box detection errors in Section 4.3.1; they are not treated as an additional semantic class here.

Table 4.10: Object-class assignment conditional on successful IoU association, pooled across the four dynamic datasets.

Class	Associated objects	Precision	Recall	F1
Bin	2,639	1.000	1.000	1.000
Box	28,076	0.996	0.999	0.998
Bucket	6,346	0.996	0.983	0.989
Macro average	37,061	0.997	0.994	0.995

The conditional class accuracy is 0.996: 36,924 of 37,061 associated objects are on the confusion-matrix diagonal, leaving 137 class substitutions. The largest substitution is 110 buckets labelled as boxes. This high conditional accuracy does not cancel the weaker bounding-box recall: an object that was never localized cannot enter the conditional class-assignment evaluation.

YOLO object-class assignment, conditional on IoU-matched localization

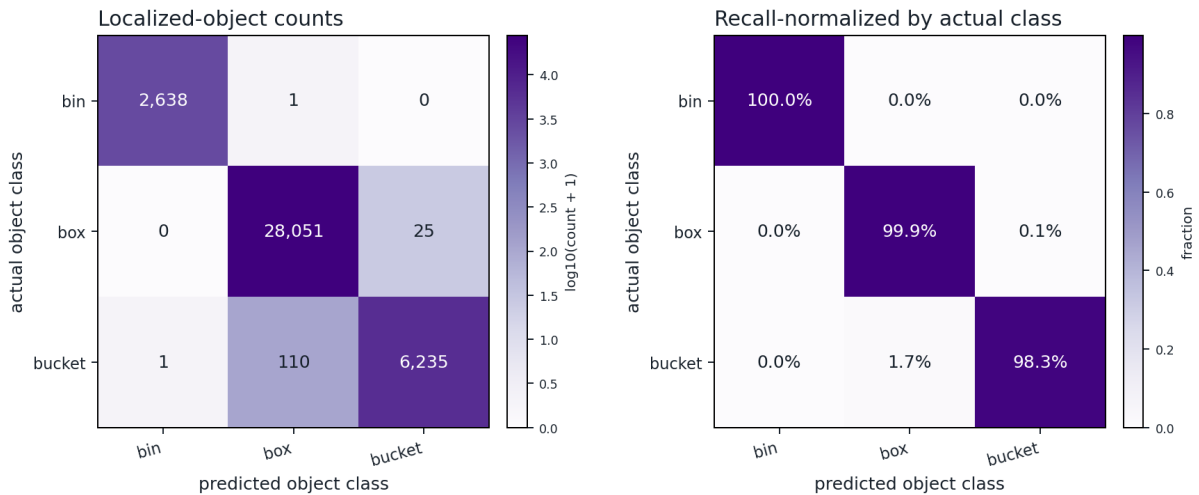


Figure 4.5: Object-class confusion conditional on class-agnostic IoU association. Rows are actual object classes and columns are predicted object classes. The left panel gives counts on a logarithmic colour scale; the right panel normalizes within each actual class. Background is intentionally excluded.

4.4 Observed Anomaly Catalogue

Table 4.11 records anomalous behaviour visible in the logs and derived metrics. It deliberately separates observation from causal explanation: no item below identifies a root cause, and the possible mechanisms listed in the analysis catalogue remain untested.

Table 4.11: Anomaly catalogue for the relogged campaign. Severity prioritizes impact on interpretation, not certainty about cause.

ID	Severity	Observation	Quantitative evidence and reporting consequence
A1	Critical	Requested VINS+YOLO filtering was inactive	All 12 folders contain 28,038 mask rows in total, but zero matched detector messages, positive-box rows, or nonzero-coverage rows. They cannot measure an active semantic-filter treatment.
A2	High	ORB+YOLO exposure varied between repeats	Positive-box fractions range from 1.4% to 99.2%, including wide variation among repeats of the same bag. A matched detector message can contain zero boxes, so match count alone is not treatment dose.
A3	Critical	Every dynamic trajectory contains discontinuities	All 48 dynamic runs contain impossible-speed steps. Median affected-step fractions are 59.6% (VINS), 30.3% (ORB), 61.1% (requested VINS+YOLO), and 26.9% (ORB+YOLO). Global ATE cannot be read as ordinary drift.
A4	High	Global and clean-segment errors are strongly separated	Median global-to-clean ATE ratios are 100, 146, 101, and 239 respectively. Locally coherent intervals coexist with catastrophic full-run discontinuities.
A5	Medium	Static ORB has an isolated initialization-adjacent jump	All nine static ORB runs have one or two jumps, with the first near 2.6 s. Where an inertial transition timestamp exists, the jump precedes it by 0.10–0.17 s; this temporal association does not establish causation.
A6	Medium	Two ORB run summaries are incomplete	The two static repeat summaries omit final inertial/map fields, although frame and local-mapping logs record initialized operation. Summary-only validation would falsely exclude these runs.
A7	High	ORB baseline/filter inputs are not paired in 11 of 12 cases	Invalid pairs differ in received-frame count by 9.95–30.06%, while their durations differ by less than 0.13%. Most between-mode differences are descriptive, not controlled treatment effects.
A8	Medium	VINS reset counts are non-discriminating	All 33 VINS baseline and requested-filter runs have failure detection disabled and record zero resets. Zero resets therefore cannot establish healthy estimation.

4.5 Trajectory Accuracy, Plausibility, and Robustness

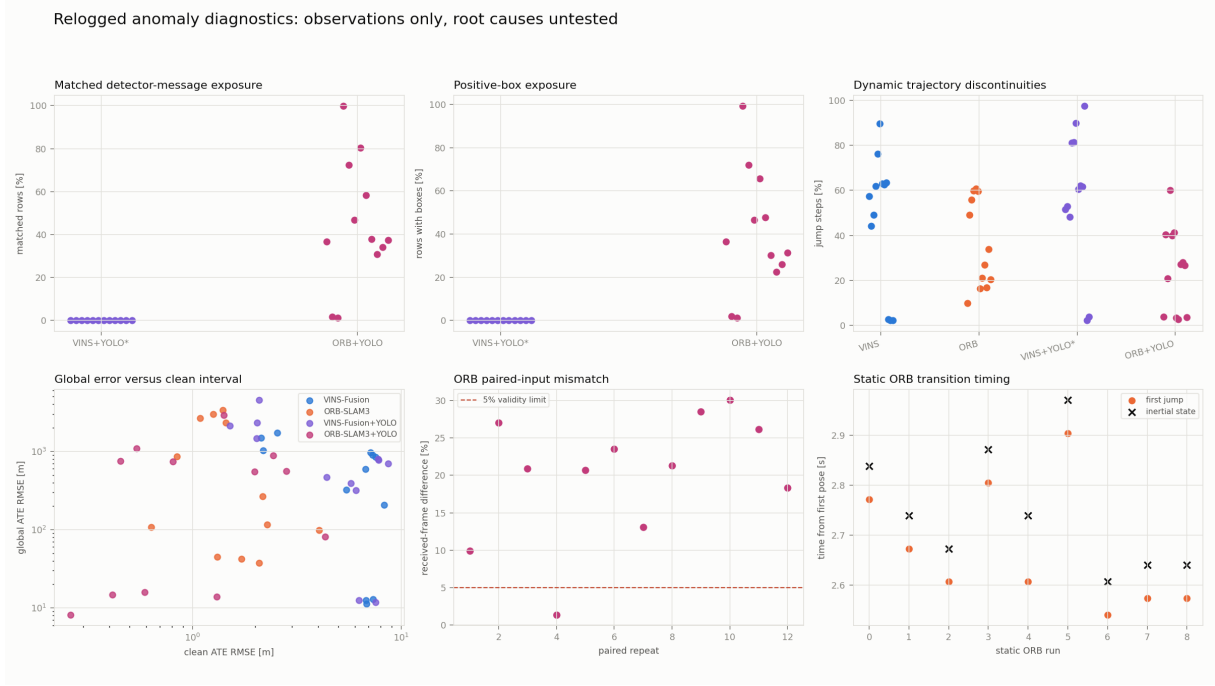


Figure 4.6: Run-level anomaly diagnostics. The panels show detector-message and positive-box exposure, dynamic jump fractions, global versus clean-segment ATE, ORB paired-input mismatch, and static ORB jump timing. These are observations only; the plot does not assign a root cause.

[Excluded: dynamic-scene trajectory results] Every dynamic-scene trajectory result in this section, and the anomalies A3 and A4 that derive from them, are **withheld as findings** and retained only as excluded evidence. The reason is the environment, not the measurement: the dynamic worlds move their props along scripted kinematic paths. The props carry collision geometry but produce no dynamic response against the robot, nothing constrains their speeds or routes to warehouse traffic, and they can occupy the same space as the vehicle without consequence. Numbers measured there describe the simulation, not the problem the thesis poses.

The runs happened, the artifacts are intact, and the plausibility failures are real properties of those recordings, so they are reported below rather than deleted — an excluded measurement that explains a methodological correction is worth more in the record than a gap. What must not be done is to read them as evidence about estimator behaviour in a dynamic warehouse.

Two consequences follow. The condition this work is named for currently has no valid testbed: it is untested, not tested and failed. And the environment becomes the leading candidate explanation for the breakdown itself, displacing the open question left in section 4.4 — testable by rebuilding the worlds with physically simulated props and re-running, which requires no change to either estimator.

Unaffected: the static-scene results here, the estimator comparison of section 4.2, the detector evaluation of section 4.3 — prop motion is kinematic, which changes contact physics rather than appearance, so detectability and class assignment are untouched — and the map evaluation of section 4.7.

Table 4.12 reports medians and IQRs because the dynamic-run distributions are strongly skewed. In static scenes, VINS attained a median translational ATE RMSE of

0.370 m and rotational RMSE of 1.56° over nine valid runs. The nine valid static ORB runs attained 0.454 m and 17.92° . The orientation difference is much larger than the positional difference and must be retained as a separate result. [Section 4.5.1](#) shows that it is produced almost entirely by one or two discrete events per run rather than by continuously worse orientation tracking.

Table 4.12: Trajectory error against rosbag ground truth, median (IQR). “Clean” is ATE RMSE after independently aligning the longest segment without an implied speed above 10 m s^{-1} . Requested VINS+YOLO values are not active-filter measurements.

Scene	Requested mode	ATE RMSE (m)	Rotation RMSE (deg)	Clean ATE (m)	Jumps
Static	VINS-Fusion	0.370 (0.063)	1.56 (0.98)	0.370 (0.063)	0 (0)
Static	ORB-SLAM3	0.454 (0.157)	17.92 (1.72)	0.341 (0.151)	1 (1)
Dynamic	VINS-Fusion	724.13 (823.52)	53.97 (39.38)	6.80 (2.57)	1116 (1157.75)
Dynamic	ORB-SLAM3	190.48 (2315.44)	108.29 (116.13)	1.42 (0.89)	484 (418.5)
Dynamic	VINS-Fusion+YOLO*	738.63 (1251.84)	56.50 (37.88)	5.91 (5.53)	1559.5 (717)
Dynamic	ORB-SLAM3+YOLO	556.05 (769.39)	138.42 (60.34)	1.06 (1.58)	208 (225.5)

*Filter requested but no detection frame was matched.

Every dynamic run contains at least one impossible-speed step. Nine of 12 baseline VINS runs, all 12 baseline ORB runs, 10 of 12 nominal VINS+YOLO runs, and all 12 ORB+YOLO runs also exceeded the reference-free mean-speed limit of 3 m s^{-1} . Peak translational error exceeded 50 m in 9, 12, 10, and 10 runs respectively. Global ATE is therefore dominated by trajectory breakdown. The clean-segment ATE is substantially smaller, which an earlier draft read as evidence that the runs contain locally coherent intervals. [Section 4.5.1](#) withdraws that reading: measured directly, the jump-free relative error is 1.8 m to 3.8 m per second, so the estimate is locally wrong by more than the distance the robot travels. Aligning a short segment absorbs much of that error, which is why clean-segment ATE looked reassuring.

The per-repeat values behind those aggregates are given in [table 4.15](#).

4.5.1 Relative pose error

RPE was implemented after the reporting cutoff and applied to the campaign without re-running any estimator; [section 3.9.1](#) defines it. It is reported here because it needs no global alignment, so unlike ATE it cannot be distorted by a single discontinuity re-seating the rigid fit. [Table 4.13](#) gives the headline interval.

Table 4.13: Relative pose error at $\Delta = 1 \text{ s}$, median (IQR) over runs. “Jump-free” excludes pairs spanning a detected tracking jump; the difference between the two columns is the teleport contribution. Requested VINS+YOLO values are not active-filter measurements.

Scene	Requested mode	Jumps	Translational (m)		Rotational (deg)	
			all pairs	jump-free	all pairs	jump-free
Static	VINS-Fusion	0 (0)	0.063 (0.027)	0.063 (0.027)	0.57 (0.05)	0.57 (0.05)
Static	ORB-SLAM3	1 (1)	0.310 (0.308)	0.096 (0.064)	10.56 (0.69)	0.25 (0.33)
Dynamic	VINS-Fusion	1116 (1158)	44.652 (49.397)	2.825 (1.595)	2.05 (1.63)	2.76 (2.01)
Dynamic	ORB-SLAM3	484 (418)	19.477 (158.567)	2.079 (0.267)	15.15 (4.82)	2.49 (2.51)
Dynamic	VINS-Fusion+YOLO*	1560 (717)	44.106 (95.564)	3.784 (1.611)	2.33 (4.76)	2.72 (1.02)
Dynamic	ORB-SLAM3+YOLO	208 (226)	38.174 (78.432)	1.805 (0.634)	21.60 (9.72)	6.33 (10.39)

Two results follow, and the first corrects this chapter.

The dynamic runs are not locally coherent. Excluding every pair that spans a jump, dynamic translational RPE at $\Delta = 1$ s is still 1.8 m to 3.8 m. The robot covers about 1.4 m in that second, so between teleports the estimate misstates its own motion by more than the distance travelled. Against the static runs, whose jump-free figures are 0.063 m to 0.096 m, dynamic local error is 20 to 60 times worse. The corruption is therefore continuous, not confined to isolated steps, and the earlier reading of clean-segment ATE as evidence of local coherence does not survive. This was a registered prediction before the metric was run, and it failed in the direction that matters.

Table 4.14: Jump-free translational RPE against interval, median over runs, in metres. “Growth” is the ratio from 0.5 s to 5 s; linear accumulation would give 10 $\times$ and a pure random walk 3.2 $\times$.

Scene	Requested mode	0.5 s	1.0 s	2.0 s	5.0 s	Growth
Static	VINS-Fusion	0.038	0.063	0.112	0.252	6.7 $\times$
Static	ORB-SLAM3	0.057	0.096	0.162	0.307	5.4 $\times$
Dynamic	VINS-Fusion	1.524	2.825	5.067	10.082	6.6 $\times$
Dynamic	ORB-SLAM3	1.158	2.079	3.776	6.787	5.9 $\times$
Dynamic	VINS-Fusion+YOLO*	2.189	3.784	5.455	10.985	5.0 $\times$
Dynamic	ORB-SLAM3+YOLO	0.976	1.805	3.041	4.384	4.5 $\times$

Table 4.14 shows why this is one mechanism rather than two. Error grows with interval at 4.5 to 6.7 $\times$ over a tenfold increase in Δ , and that growth exponent is the same for static and dynamic scenes; only the magnitude differs, by roughly 40 $\times$. The dynamic runs are not failing in a qualitatively different way from the static ones — they are running the same error process far hotter.

ORB-SLAM3’s static rotational penalty is entirely jump-localised. Static ORB-SLAM3 carries a rotational ATE of 17.92° against VINS-Fusion’s 1.56°, an elevenfold gap that invites the reading that ORB-SLAM3 tracks orientation poorly. RPE refutes it. Across the nine static ORB runs, all-pairs rotational RPE is 9.28° to 11.63°, but jump-free rotational RPE is 0.10° to 0.66° — at or below VINS-Fusion’s 0.49° to 0.69° on the same recordings. Every one of those runs contains one or two jumps and no more.

The whole rotational gap is thus produced by one or two discrete re-seating events per run, not by continuously worse tracking. This gives anomaly A5 its significance: that jump occurs near 2.6 s, immediately before inertial initialization completes, and it is predominantly rotational. Between those events ORB-SLAM3’s orientation tracking is competitive with VINS-Fusion’s. No causal mechanism is established here — the association is temporal — but the error is now localized to an event rather than attributed to the estimator’s steady-state behaviour.

Baseline ORB-SLAM3 logged 7.75 ± 6.03 tracking-loss events and 9.52 ± 6.49 s of lost tracking per dynamic run (mean $\pm$ sample SD, $n = 12$). ORB+YOLO logged 9.17 ± 3.01 events and 11.46 ± 5.32 s. VINS logged no estimator resets in either requested mode, but the absence of resets does not make its trajectories plausible. The event types are estimator-specific and are not treated as numerically equivalent.

Table 4.15: Truth-based trajectory error for every repeat of the relogged campaign. Every dataset is a no-floor-texture world, so that shared part of each dataset name is omitted. “ATE” and “clean” are global and longest-clean-segment ATE RMSE in metres, “rot” is rotational geodesic RMSE in degrees, and “jmp” counts steps implying more than 10 m s^{-1} . All 66 runs are operational; two static ORB run summaries have incomplete schemas, and only one of 12 ORB baseline/filter pairs passed input matching. Static datasets were not run in either filtered mode; those cells are set to –.

Dataset	Rep	VINS-Fusion					ORB-SLAM3					VINS-Fusion+YOLO*					ORB-SLAM3+YOLO				
		ATE (m)	rot (deg)	clean (m)	jmp (n)		ATE (m)	rot (deg)	clean (m)	jmp (n)		ATE (m)	rot (deg)	clean (m)	jmp (n)		ATE (m)	rot (deg)	clean (m)	jmp (n)	
dynamic 00_000	01	596.37	38.5	6.77	858	45.21	176.5	1.31	109	392.55	107.0	5.74	1122	13.84	76.7	1.31	37				
dynamic 00_000	02	206.97	92.2	8.32	609	857.95	162.0	0.84	512	466.72	108.0	4.39	1183	892.31	129.9	2.45	213				
dynamic 00_000	03	324.01	88.4	5.47	580	3364.34	70.8	1.40	610	318.66	108.7	6.07	991	81.20	173.1	4.33	119				
dynamic 00_001	01	1030.40	63.6	2.18	1374	3002.57	29.7	1.26	1150	2309.93	74.6	2.04	1805	2885.77	82.2	1.42	1159				
dynamic 00_001	02	1481.00	62.8	2.14	1695	2669.73	26.1	1.09	1141	2129.97	75.2	1.51	1810	553.35	93.5	1.98	454				
dynamic 00_001	03	1724.94	175.1	2.55	1995	2311.30	23.1	1.45	1098	1457.89	59.6	2.03	1997	558.76	112.8	2.82	378				
dynamic 01_000	01	904.94	53.3	7.29	1608	98.88	169.1	4.05	239	698.15	52.0	8.67	1545	8.20	102.3	0.26	44				
dynamic 01_000	02	851.90	53.8	7.55	1596	115.45	172.4	2.28	327	796.03	52.6	7.77	1582	14.78	173.8	0.41	26				
dynamic 01_000	03	965.82	54.1	7.14	1619	107.60	167.7	0.64	456	779.10	53.3	7.79	1574	749.16	162.6	0.45	203				
dynamic 02_000	01	12.49	6.1	6.80	72	37.83	59.3	2.09	388	4518.77	25.1	2.09	2610	1097.95	159.7	0.54	231				
dynamic 02_000	02	12.88	5.5	7.36	61	265.50	62.9	2.17	689	11.85	6.4	7.55	62	737.27	148.9	0.81	219				
dynamic 02_000	03	11.30	4.3	6.81	58	42.37	145.8	1.72	408	12.55	6.9	6.29	66	15.81	146.9	0.59	31				
static 000	01	0.37	1.6	0.37	0	0.58	17.9	0.55	1	–	–	–	–	–	–	–	–				
static 000	02	0.41	1.5	0.41	0	0.79	20.4	0.77	1	–	–	–	–	–	–	–	–				
static 000	03	0.43	1.6	0.43	0	1.20	19.1	0.30	2	–	–	–	–	–	–	–	–				
static 001	01	0.39	2.3	0.39	0	0.45	18.9	0.27	2	–	–	–	–	–	–	–	–				
static 001	02	0.29	1.8	0.29	0	0.45	18.1	0.21	2	–	–	–	–	–	–	–	–				
static 001	03	0.48	2.2	0.48	0	0.38	17.2	0.26	1	–	–	–	–	–	–	–	–				
static 002	01	0.34	0.8	0.34	0	0.42	16.7	0.39	1	–	–	–	–	–	–	–	–				
static 002	02	0.36	0.8	0.36	0	0.46	17.6	0.43	1	–	–	–	–	–	–	–	–				
static 002	03	0.34	0.7	0.34	0	0.36	16.9	0.34	1	–	–	–	–	–	–	–	–				

*Filter requested but no detection frame was matched in any repeat; these columns are not active-filter measurements.

Requested mode	Poses (n)	Overlap (s)	Path (m)	ATE RMSE (m)	Rot RMSE (deg)	Clean ATE (m)	Jumps (n)
VINS-Fusion	1494	77.0	5486.93	596.367	38.51	6.771	858
ORB-SLAM3	1105	77.3	408.22	45.206	176.53	1.314	109
VINS-Fusion+YOLO*	2175	77.1	1958.14	392.546	107.04	5.743	1122
ORB-SLAM3+YOLO	943	77.2	239.53	13.839	76.72	1.310	37

*Filter requested but no detection frame was matched.

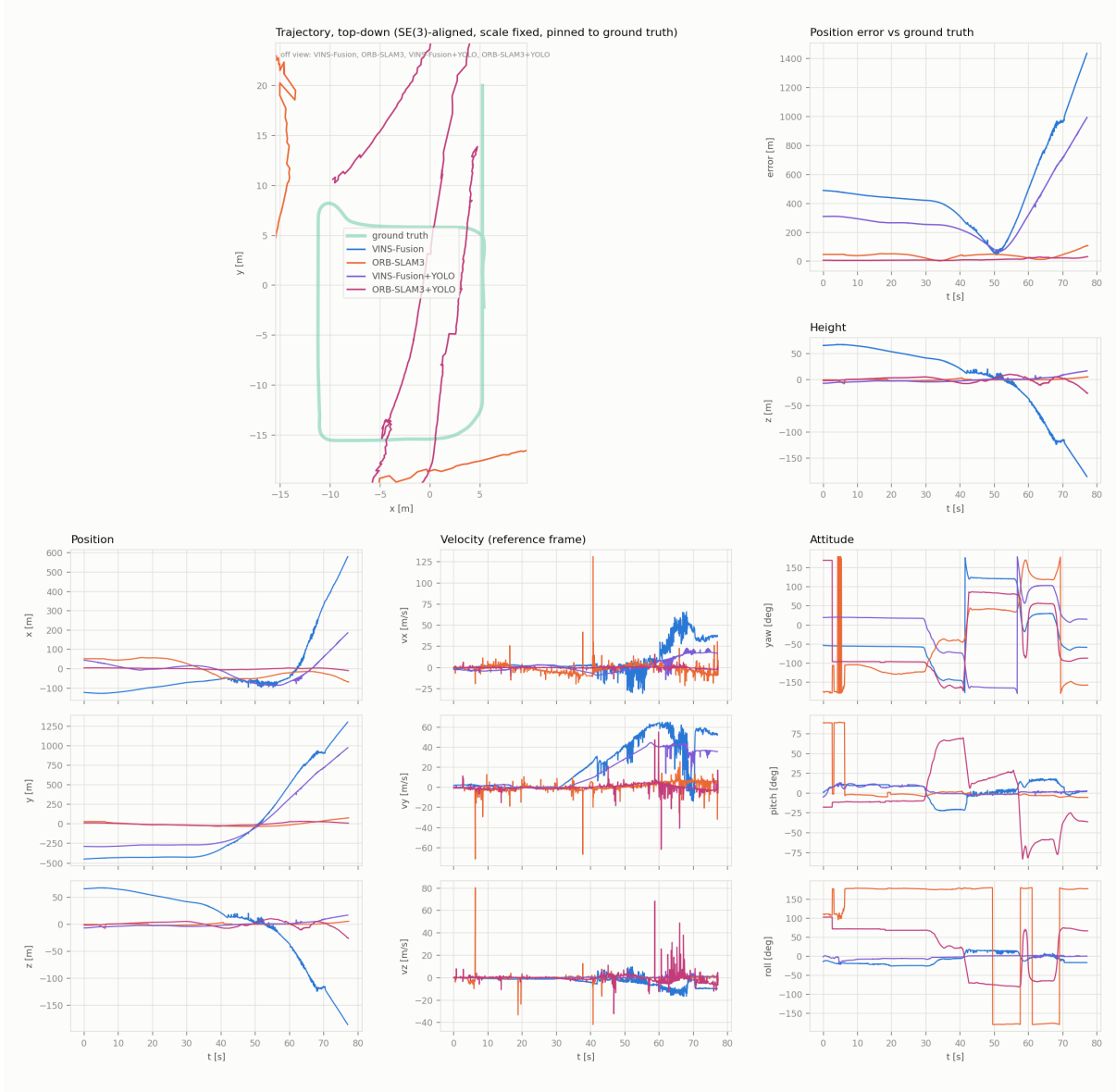


Figure 4.7: Representative repeated-run trajectory diagnostic for dataset_dynamic_nof loortexture_00_000/logging_20260916_01, in the same visual format as the existing comparison figures. The table gives the per-system summary for this run; “clean” is ATE RMSE over the longest jump-free segment. The very large paths and frequent jumps are retained rather than clipped into a favourable view. ATE and rotational errors use the rosbag /ground_truth/odometry topic after rigid $SE(3)$ alignment with scale fixed. The nominal VINS+YOLO trace is invalid as a filtering treatment because no masks were matched.

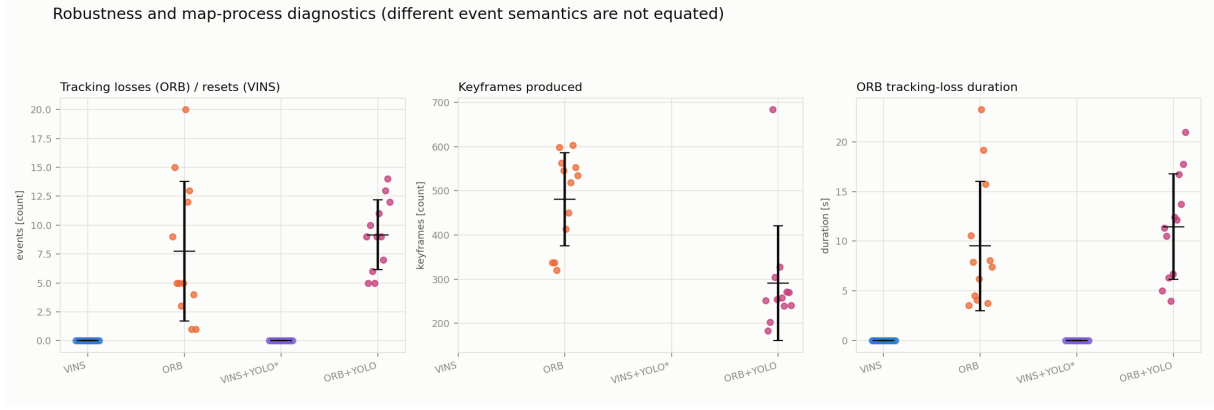


Figure 4.8: Per-run robustness and keyframe observations for the dynamic datasets. Points are experimental runs and black marks are mean $\pm$ sample SD. VINS reset count and ORB tracking-loss count have different semantics. The asterisk identifies the requested VINS filter mode in which no masks were matched.

4.6 Real-Time and Computational Measurements

Table 4.16 summarizes process-local measurements over the 12 dynamic runs in each requested mode. CPU is the change in cumulative process CPU time divided by monotonic wall time; 100% denotes one fully used logical core, so values above 100% represent multithreaded execution. Memory is peak resident set size. The separate detector process, rosbag, Gazebo, RViz, and RTAB-Map are not included.

Table 4.16: Dynamic-run capacity and resource measurements, mean $\pm$ sample SD, $n = 12$ per row. Nominal VINS+YOLO values are not active-filter measurements.

Requested mode	Throughput (Hz)	Retained input (%)	CPU (% core)	Peak RSS (MiB)
VINS-Fusion	27.14 ± 5.81	100.00 ± 0.00	443.66 ± 58.96	629.51 ± 282.62
ORB-SLAM3	23.47 ± 4.77	88.55 ± 13.42	385.22 ± 27.71	1226.06 ± 81.83
VINS-Fusion+YOLO*	28.83 ± 3.16	100.00 ± 0.00	406.43 ± 89.95	646.14 ± 306.15
ORB-SLAM3+YOLO	14.47 ± 5.42	66.96 ± 15.26	410.86 ± 42.83	1054.64 ± 91.11

*Filter requested but no detection frame was matched.

The ORB+YOLO folders show lower achieved throughput and retention than baseline ORB, while process CPU is of similar order. This is an observation, not an isolated detector-cost effect: the external detector is excluded from CPU and 11 paired runs received materially different image counts. Baseline VINS processed all enqueued images in its logs, whereas baseline ORB discarded frames whenever the one-frame queue was overwritten.

Per-module results use each run’s event distribution. Figure 4.10 shows pooled event boxplots for shape, overlaid with one median point per run so frames are not mistaken for independent experimental repeats. Across run medians, ORB tracking was 63.19 ± 7.06 ms and ORB local mapping was 189.58 ± 16.95 ms. The requested ORB+YOLO mode measured 112.53 ± 24.61 ms and 356.30 ± 68.36 ms respectively. VINS feature tracking was 48.00 ± 10.83 ms and its back end was 60.41 ± 17.12 ms; nominal VINS+YOLO values were 43.61 ± 11.49 ms and 57.90 ± 16.29 ms, but cannot be interpreted as filter effects because that filter was inactive. These timings refer to different estimator stages and are not added or treated as directly equivalent algorithms.

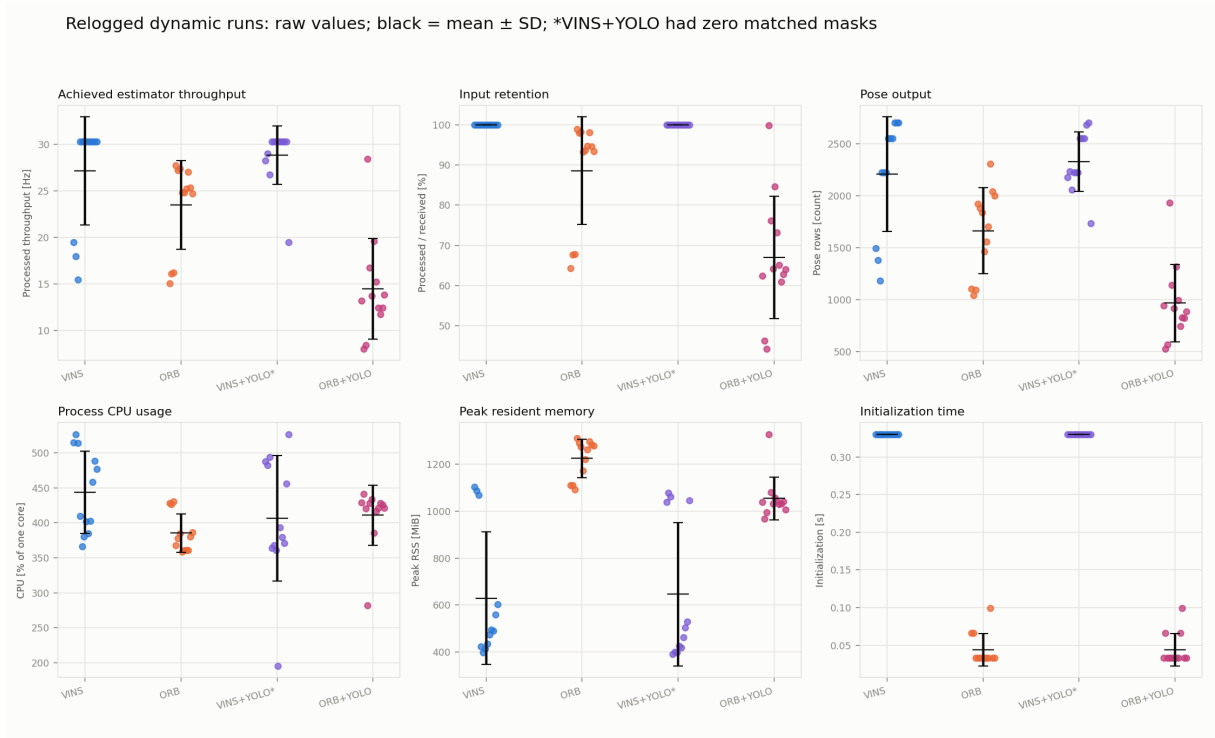


Figure 4.9: Raw dynamic-run capacity and process-resource values. Each point is a run; black marks show mean $\pm$ sample SD. This plot exposes run-to-run spread rather than presenting only aggregate bars.

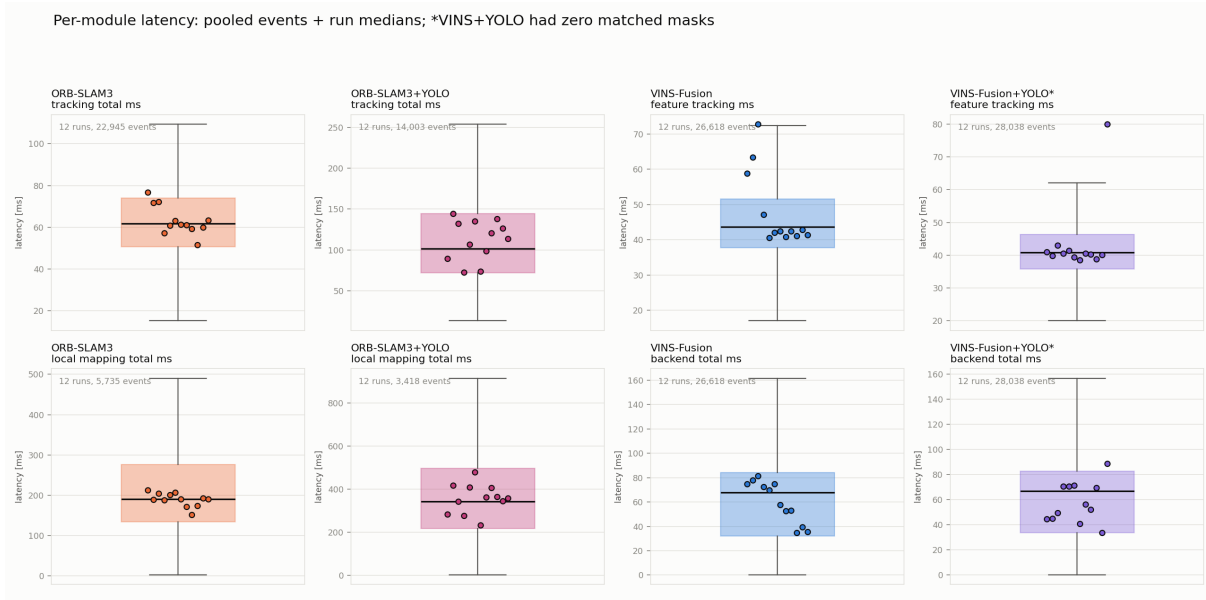


Figure 4.10: Module-latency distributions for dynamic datasets. Boxes summarize all recorded module events; coloured points are per-run medians ($n = 12$ in each requested mode). ORB local mapping is asynchronous per keyframe, while VINS back-end rows follow estimator updates.

4.7 Dense Map Reconstruction and Map-Quality Evaluation

4.7.1 Question, scope, and conditions

Trajectory error answers whether the robot knew where it was. It does not answer whether the map it produced describes the building, nor whether a planner could have used that map. This section evaluates the one dense map reconstructed so far against two references of very different strength: the simulator’s own collision geometry, which is independent of the map, and the ground-truth route of the run that built the map, which is not. The two questions are kept separate because they fail in different ways — a map can be geometrically faithful and still unusable, and it can be navigable over an easy route while misrepresenting the parts of the building that route never entered — and because only the first reference is independent evidence.

The evaluation is confined to the **tiny** warehouse. A large-warehouse equivalent is **planned**; no database for it exists at the reporting cutoff. Tiny-map and large-map results must not be pooled or read as repeated trials of one condition, because the two worlds differ in extent, aisle width, and route length. Table 4.17 records the conditions.

Table 4.17: Conditions for the dense-map evaluation. The database and the ground-truth route come from the *same* run, which bounds what the drivability figures can establish.

Factor	Level
Map under test	map/map_tiny.db, RTAB-Map 0.23.7, created 2026-09-08
World	aws-robomaker-small-warehouse-world/worlds/small_warehouse_static/small_warehouse_static_tinymap.world (tiny geometry, despite residing in the small_warehouse_static family directory)
Scene dynamics	static
Grid resolution	0.05 m
Geometry tolerance	0.25 m
Robot footprint	0.83 × 0.50 m rectangle from nav2_ackermann.yaml, not a circumscribed radius
World-to-map placement	$x = 1.850$ m, $y = 8.600$ m, $\theta = -89.90^\circ$, measured once by script/fit_map_pose.py
Drivability reference, as plotted	dataset_static_tiny_repeat, 2973 poses; an independent run, no longer present in the workspace or on the removable drive
Substitute available	dataset/dataset_map_tiny, /ground_truth/odometry: 3032 messages over 60.6 s of simulation time. This is the run that <i>built</i> the map, so it is not an equivalent replacement
Evaluator	script/validate_map.py
Experimental unit, n	one database; $n = 1$

4.7.2 Map placement as a validity precondition

Both metrics depend on seating the occupancy grid in world coordinates, so placement is a precondition rather than a display choice: a mis-seated grid scores real structure against the wrong cells and makes a good map look bad. The transform is a constant for

a given database and is measured once, by fitting occupied cells to the world file’s wall and obstacle edges without using odometry or ground truth. It is deliberately not taken from the live `map` $\rightarrow$ `odom` correction, which moves with front-end drift.

The measured fit is (1.850, 8.600) m at -89.90° , with a margin of 60.9 % over the best rival orientation more than 5° away. An independent prediction — the spawn pose of the IMU body frame, which sits 0.338 m ahead of `base_footprint` — gives (1.800, 8.662) m. The two agree to 0.080 m, inside the grid’s own 0.05 m resolution. Two independent routes to the same number is what makes this a measurement rather than a tuned parameter.

4.7.3 Graph and database summary

Table 4.18 summarizes the stored graph, read directly from the database in read-only mode. The counts are small: 30 nodes over a 59.3 s span, reflecting RTAB-Map’s retention policy rather than the rate at which images were offered to it.

Table 4.18: Stored contents of `map/map_tiny.db`. Link rows are stored per direction; the undirected count is the number of distinct node pairs.

Element	Rows	Undirected	Note
Nodes	30	–	ids 1–30, stamp span 59.3 s
Neighbour links	56	28	sequential odometry constraints
Global loop closures	5	3	pairs (9, 22), (10, 23), (29, 30)
Gravity links	30	30	one per node
Local-space links	0	0	none recorded
Visual words	10 357	–	vocabulary size
Features	28 374	–	stored keypoints

Of the three distinct closure pairs, (9, 22) and (10, 23) join nodes roughly half a lap apart and are genuine revisits of an earlier place. The third, (29, 30), joins consecutive nodes and is therefore not a revisit; it should not be counted as evidence of place recognition.

4.7.4 Geometry against the world file

Geometry is scored against the rasterized collision geometry of the world file at a 0.25 m tolerance. Recall is measured against *surfaces* rather than filled interiors, because a stereo camera can only observe the outside of a shelf; counting its interior as missed geometry would cap even a perfect map well below 100 %.

Table 4.19: Occupied-cell geometry of `map/map_tiny.db` against `small_warehouse_static_tinyworld.world` at 0.25 m tolerance.

Metric	Value	Population
Occupied-cell precision	96.6 %	11 634 occupied cells
Recall, region the map observed	89.5 %	observed surfaces
Recall, whole building	67.2 %	all surfaces in the world file

The gap between 89.5 % and 67.2 % is route coverage, not mapping failure: the run never entered the side aisles, so roughly a third of the building was never observed by any sensor. The two numbers answer different questions and neither replaces the other — the first is a statement about mapping quality, the second about where the robot went.

4.7.5 Drivability against the true route

The robot demonstrably drove the reference route, so every pose on it was collision-free in reality. Stamping the real footprint onto the map at each pose therefore converts any collision into a *proven* false obstacle, and any unknown cell into a hole a planner would have to cross blind. This metric needs no assumption about what a good map looks like.

It does, however, depend on where the route came from, and that is the weak point of this result. The route used to produce [table 4.20](#) came from a separate run, `dataset_static_tiny_repeat`, which is no longer present in the workspace or on the removable drive. The figures below therefore stand as a dated independent measurement that cannot currently be re-derived.

The only surviving recording of this warehouse, `dataset/dataset_map_tiny`, is not an equivalent substitute, because it is the run that built the map. Three measurements establish this. Applying the placement of [section 4.7.2](#) to the stored node poses puts them a median of 0.069 m from that recording’s ground-truth route; comparing each node against the ground-truth pose carrying the *same* timestamp gives a median of 0.340 m and a maximum of 0.52 m; and all 30 nodes fall inside the recording’s sim-time interval. The second of these is the discriminating one, since a different run over the same loop would agree in space but not in time. The two routes are also visibly different: the lost run is a single oval circuit, while this one is a figure-eight through the centre of the building.

The evaluation was nevertheless repeated against this recording, and the outcome confirms the objection rather than resolving it. Both results are reported in [table 4.20](#).

Table 4.20: Drivability of `map/map_tiny.db` under the true 0.83×0.50 m nav2 footprint, evaluated against two different routes. “Independent” is `dataset_static_tiny_repeat`, the run that did not build the map and is no longer available. “Build run” is `dataset/dataset_map_tiny`, the run the map was constructed from. Geometry is identical in both cases because it does not depend on the route.

Metric	Independent	Build run	Meaning
Poses scored	2973	3032	
Clear and mapped	99.3 %	100.0 %	footprint free and observed
False obstacle	0.7 %	0.0 %	proven error
Unmapped under footprint	0.0 %	0.0 %	blind cells on the route
Clearance, median	1.07 m	1.01 m	to nearest obstacle
Clearance, 5th percentile	0.45 m	0.46 m	inflation radius is 0.80 m

The difference between the two columns is the independence effect, measured rather than assumed. Against the run that built it, the map records *no* false obstacle at all: it cannot contradict the trajectory it was constructed from. The 0.7 % that the independent run exposes is precisely the part of the map that only a route the mapper had not seen could reveal. The right-hand column is therefore not a better result than the left; it is a weaker question, and the 0.0 % should not be quoted as a map-quality figure.

Clearance is nearly unchanged between the two, which is expected: it is a property of where each route runs relative to mapped geometry, not of which run built the map. Both routes spend part of their length inside nav2’s inflation radius.

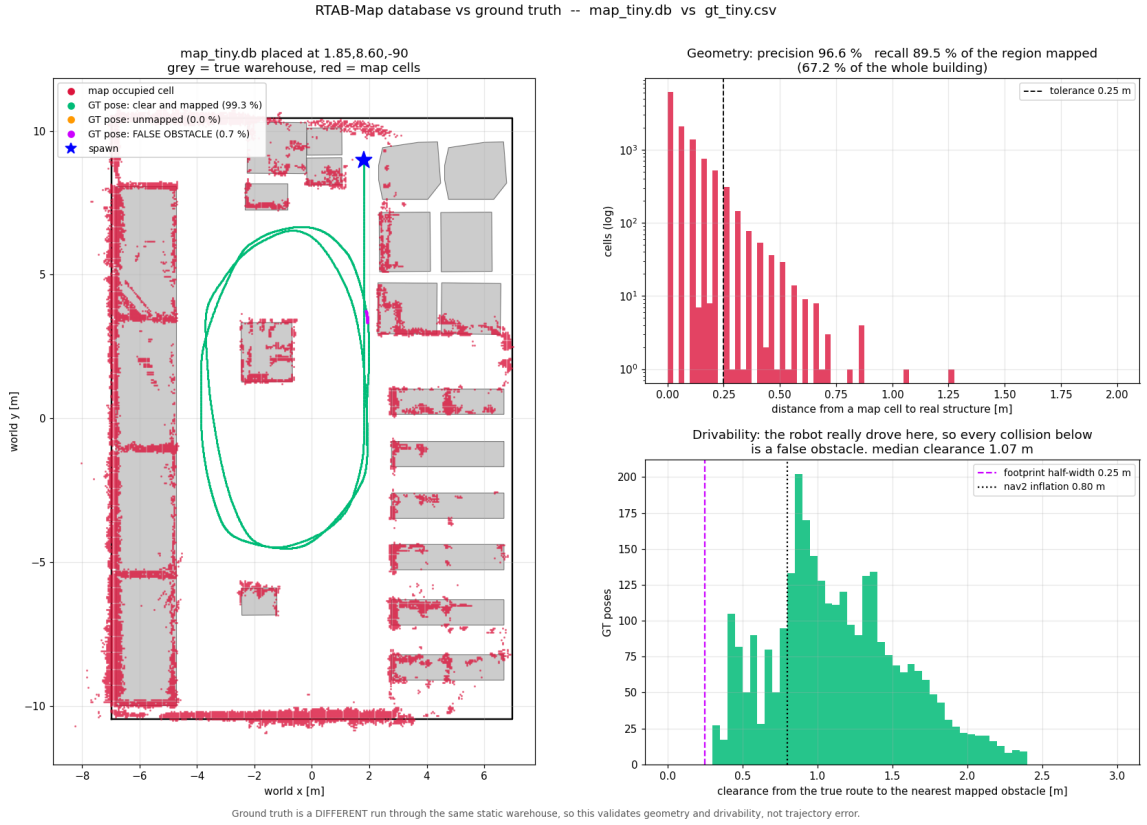


Figure 4.11: Dense-map evaluation of `map/map_tiny.db`. Left: occupied cells (red) over the world file's true collision geometry (grey), with the ground-truth route coloured by drivability and the spawn marked. Top right: distance from each occupied cell to real structure, with the 0.25 m tolerance marked; the bulk falls well inside tolerance with a thin tail to about 1.25 m. Bottom right: clearance from the true route to the nearest mapped obstacle, against the footprint half-width and the nav2 inflation radius. The ground truth was a different run through the same static warehouse, so this validates geometry and drivability, not trajectory error; that run is no longer available (section 4.7.5).

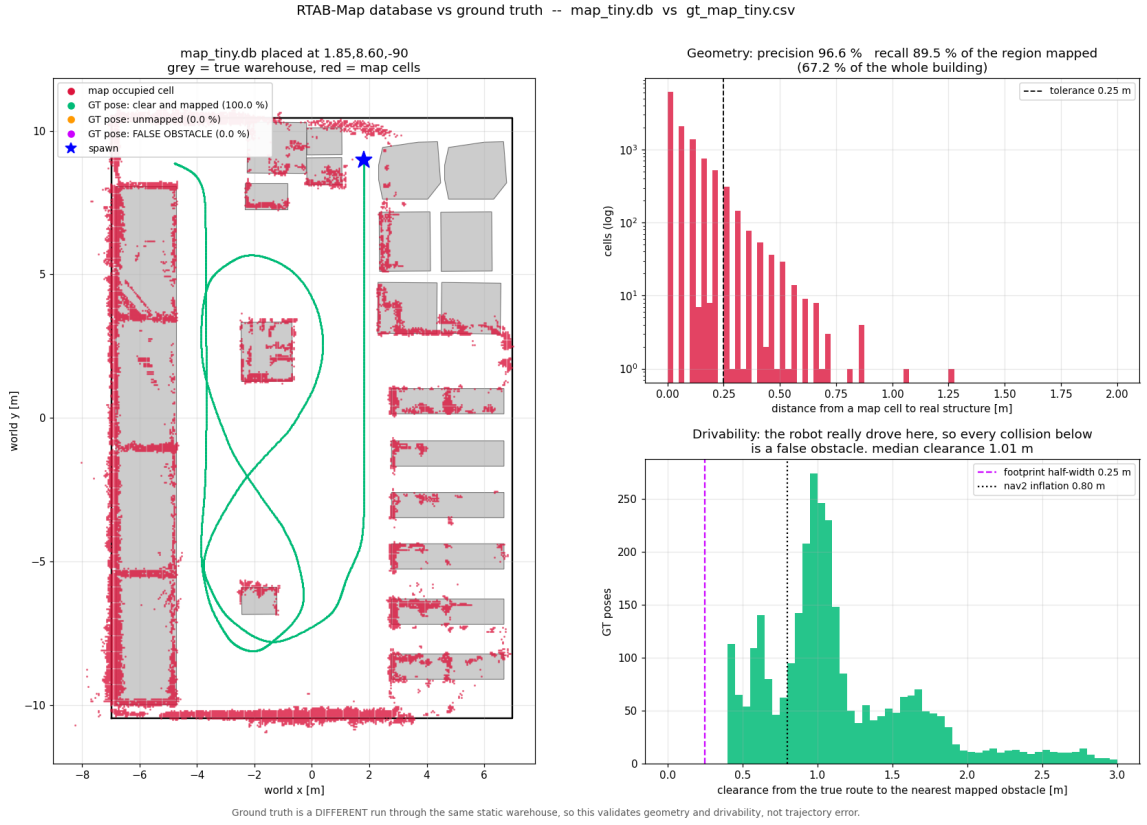


Figure 4.12: The same database scored against the run that built it, `dataset/dataset_map_tiny`. The route is a figure-eight rather than the oval of figure 4.11, and no pose on it collides with the map: the false-obstacle rate falls from 0.7 % to 0.0 %. The geometry panels are identical to figure 4.11 because geometry does not depend on the route. The footer claiming a different run is emitted unconditionally by the evaluator and is incorrect for this figure.

4.7.6 Interpretation and limitations

Within its stated scope the tiny map is geometrically faithful and navigable: 96.6 % of what it marks as occupied is real, it recovers 89.5 % of the surfaces it actually observed, and it contains no blind cells and only 0.7 % proven false obstacles along a route the robot really drove. The geometry half of that is independent evidence and is the strongest map result the project currently holds; the drivability half is a self-consistency check and is weaker than it looks. Both are a direct improvement on earlier assessments that judged the same database unnavigable — those were measured with the grid mis-seated, so the comparison was against the wrong cells. That correction is a methodological finding in its own right and is why [section 4.7](#) reports placement before any metric.

Four limitations bound what these numbers support.

- **The drivability reference cannot be re-read.** The run that provided it is lost, and the only surviving recording of this warehouse is the one that built the map, which would answer a weaker question. Re-establishing this result needs a fresh independent recording — a second run, not a second map.
- **The route is easy.** It is one large loop down the wide central aisle and never squeezes into a side aisle. The 0.7 % false-obstacle figure is therefore a floor, not a worst case. A route through the narrow aisles is the test that would actually stress this map, and it has not been run.
- **Clearance couples to planner cost.** The 5th-percentile clearance of 0.45 m sits below nav2's 0.80 m inflation radius, and a visible share of the true route lies inside it. Nothing is blocked, but a planner would charge a high cost for driving where the robot demonstrably drove. That coupling should be examined before any planned path is trusted.
- **$n = 1$, and the scope is one world.** A single database on the tiny warehouse supports no uncertainty estimate and no claim about the large warehouse, where longer routes and narrower bays change both the drift exposure and the clearance margin.
- **Residual distortion is not a placement problem.** Walls appear doubled and thickened and shelf rows bowed in the overlay. No rigid transform removes that; it is internal map distortion and is the property that would limit navigation, independently of how well the grid is seated.

[Queried: reproducibility of the drivability half] The geometry result is fully reproducible: both `map/map_tiny.db` and the world file are present. The drivability result is not. Its reference run, `dataset_static_tiny_repeat`, is absent from `dataset/` and from the removable drive, and the only surviving recording of this warehouse is the one that built the map, which would answer a weaker question ([section 4.7.5](#)). The database additionally stores an empty parameter set, so the configuration that produced it is not recoverable from the file, and the RTAB-Map engineering record's claim that the node stamps match no dataset is contradicted by the pose agreement reported above. The drivability figures should therefore be read as a dated measurement rather than a standing result, and should not be carried into the major report until an independent route has been recorded and retained.

4.7.7 Planned extension to the large warehouse

The same evaluation is **planned** for the large warehouse and is not started: no large-map database exists at the cutoff. Three things must be settled before those numbers would

mean anything. The world-to-map placement must be measured for the new database rather than assumed, since the tiny-map constant ($1.850, 8.600, -89.90^\circ$) is specific to that database and world. A ground-truth route must be recorded from a run *other* than the one that builds the map, which the tiny-map case did not do and which is what would turn drivability into independent evidence. Finally, the route should deliberately include narrow bays, so that the false-obstacle rate is exercised rather than merely reported. Until then, no map-quality claim extends beyond the tiny warehouse.

4.8 Loop Closure, Map Merging, and Global-Correction Evidence

The expanded ORB logs now expose back-end events. Baseline ORB-SLAM3 recorded six loop closures: two in each repeat of `dataset_static_nofloortexture_002`. Each of those runs also recorded two global bundle-adjustment executions. One baseline dynamic run (`dataset_dynamic_nofloortexture_02_000/logging_20260916_03`) and one ORB+YOLO run (`dataset_dynamic_nofloortexture_01_000/logging_20260916_02`) recorded a map merge. These counters demonstrate that the native ORB back-end paths executed; they do not demonstrate that the resulting corrections were correct. The failed trajectory-plausibility checks and the absence of an event-aligned before/after counterfactual prevent a correction-benefit claim. RTAB-Map reconstruction remains a separate pipeline and is not exercised by this relogged campaign; its dense-map evidence is reported independently in [section 4.7](#), on a different world and database.

4.9 Interpretation, Limitations, and Advisor Decisions

4.9.1 Where each pipeline is weak, block by block

[Figures 4.13](#) and [4.14](#) put every instrumented stage of the three pipelines on one page and colour it by a stated criterion. [Table 4.21](#) through [table 4.25](#) give the evidence behind every colour: the metric, its value, the criterion it was tested against, and which of three rules produced the verdict. Both the figures’ fills and the tables are generated by `script/block_performance.py` from the run artifacts; no colour in either figure was chosen by hand.

The three rules are deliberately distinguished, because they answer different questions and only one of them is about accuracy:

ablation

The block was removed, disabled or swapped and the run repeated. The value is the ratio of aligned ATE without it to aligned ATE with it. This is the only rule that speaks to trajectory error.

latency

The block’s median time as a share of its stage, or a stage against the 66.0ms inter-frame budget implied by 30.3Hz imagery at $0.5\times$ replay. Describes cost, not accuracy.

outcome

A counter the block emits, against a stated threshold—accepted loop candidates,

usable solver solutions, filter consistency.

Where the criteria come from. Every verdict is a comparison against a threshold, and those thresholds do not all have the same standing. The tables label each one, and the distinction matters more than the values:

derived

Follows from something outside this report. There are two. The 66.0 ms frame budget is arithmetic on the recorded conditions, $1000/(30.3 \text{ Hz} \times 0.5 \text{ replay})$: a per-frame stage slower than this cannot keep up, and no judgement is involved. The consistency target $\text{ANEES} = 3.0$ is the standard result that a correctly scaled covariance makes the normalised error squared average the degrees of freedom, which for a pose error is three.

convention

A round number adopted here so the rule is stated rather than left implicit. Nothing external fixes 40 % of a stage rather than 35 %, $2.0\times$ for a load-bearing block rather than $1.5\times$, 10 % of loop candidates accepted, or 99 % of solver calls returning a usable solution. These are contestable, which is why the measured value sits beside the threshold in every row: a reader who prefers a different line can redraw it without re-running anything.

[Withdrawn criterion] An earlier revision of these tables graded VINS-Fusion’s solver quality against “median final cost > 5000 ”. That threshold was chosen after observing 1176 in static scenes and 14110 in dynamic ones, so it separated two numbers already in hand and the “poor” verdict it produced was guaranteed by the choice rather than tested by it. Raw Ceres cost also has no absolute scale, depending on residual count and weighting, so no fixed threshold on it is meaningful in principle. It is replaced by the solver’s own `solver_solution_usable` flag, which is binary and emitted by the solver rather than invented here: 99.64 % of solves in static scenes and 99.55 % in dynamic ones return a usable solution, and the block is now graded good in both. The non-convergence rate—22.9 % of solves in static scenes and 45.3 % in dynamic ones reaching the iteration or 80 ms cap—is reported in the note column and deliberately not graded, because any threshold on it would repeat the same error.

[Reading the colours] A red block is *not* a claim that it caused the trajectory error, unless its rule is *ablation*. Only four blocks in the entire project have an ablation behind them: the EKF’s wheel correction, its IMU prediction, its yaw-rate model, and ORB-SLAM3’s loop closing. Every other red marks a block that is expensive or that emits a poor counter, which is a different and weaker statement. Yellow means no claim at all—not instrumented, not exercised in these runs, or not implemented in this fork.

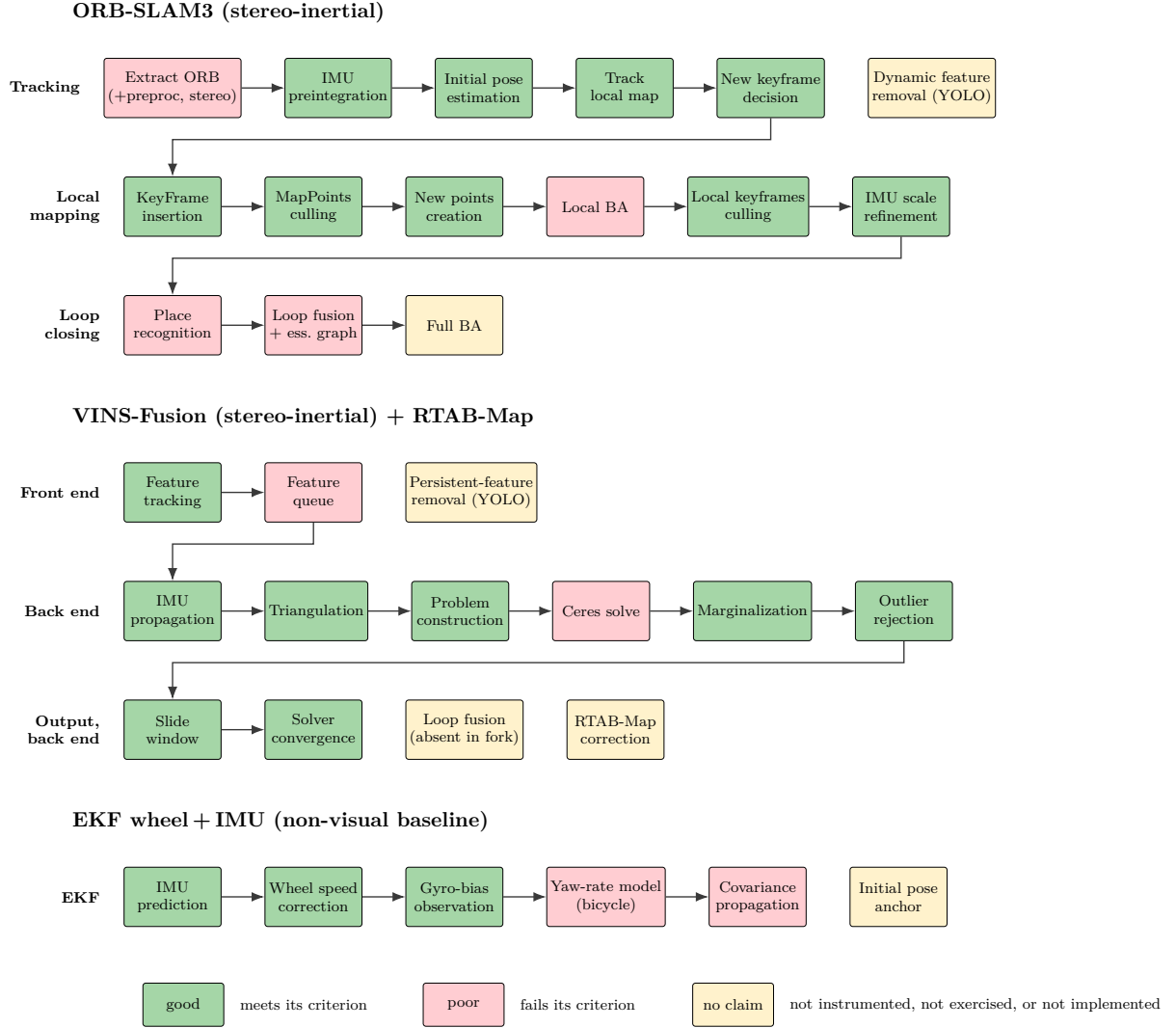


Figure 4.13: Per-block performance in the **static** condition, over the five **allsensor** datasets. The EKF baseline appears here and not in figure 4.14 because no dynamic dataset carries wheel encoders. Evidence for every colour is in tables 4.21 to 4.23, one table per algorithm.

Table 4.21: Per-block evidence for **ORB-SLAM3** in the static condition, [figure 4.13](#). Five **allsensor** datasets, median across runs, loop closing enabled (**logging_20260916_01**).

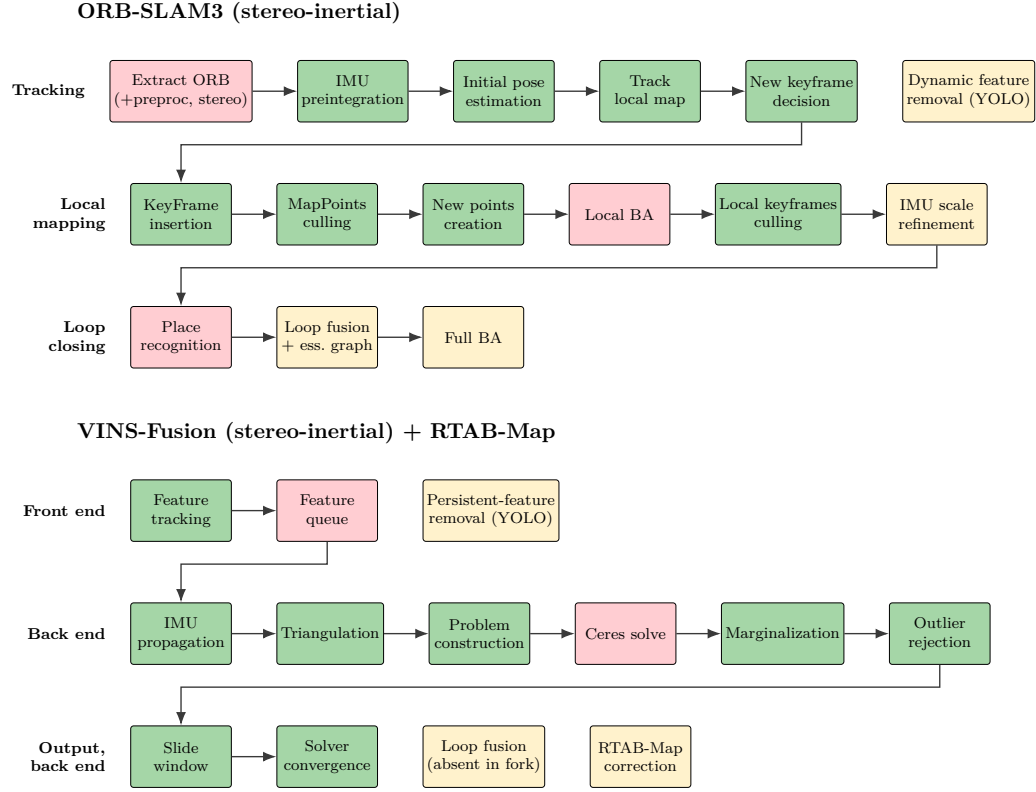
Block	Rule	Metric	Value	Criterion / note	Verdict
Tracking					
Extract ORB (+preproc, stereo)	latency	median image_preprocessing_feature_stereo_ms	33.1	<i>convention:</i> <40% of stage (61.4% of stage)	poor
IMU preintegration	latency	median imu_preintegration_ms	0.120	<i>convention:</i> <40% of stage (0.2% of stage)	good
Initial pose estimation	latency	median pose_prediction_ms	0.087	<i>convention:</i> <40% of stage (0.2% of stage)	good
Track local map	latency	median local_map_tracking_ms	16.5	<i>convention:</i> <40% of stage (30.7% of stage)	good
New keyframe decision	latency	median keyframe_decision_ms	0.169	<i>convention:</i> <40% of stage (0.3% of stage)	good
TRACKING total	latency	median tracking_total_ms	53.8	<i>derived:</i> <66.0 ms frame budget	good
Dynamic feature removal (YOLO)	none	–	–	– (filter disabled in every run reported here)	no claim
Local mapping					
KeyFrame insertion	latency	median keyframe_insertion_ms	10.3	<i>convention:</i> <40% of stage (3.3% of stage)	good
Recent MapPoints culling	latency	median map_point_culling_ms	0.383	<i>convention:</i> <40% of stage (0.1% of stage)	good
New points creation	latency	median map_point_creation_fusion_ms	41.5	<i>convention:</i> <40% of stage (13.3% of stage)	good
Local BA	latency	median local_ba_ms	244.4	<i>convention:</i> <40% of stage (78.1% of stage)	poor
Local keyframes culling	latency	median keyframe_culling_ms	10.9	<i>convention:</i> <40% of stage (3.5% of stage)	good
IMU initialization	latency	median imu_initialization_ms	0.002	<i>convention:</i> <40% of stage (negligible)	good
IMU scale refinement	latency	median inertial_refinement_ms	0.001	<i>convention:</i> <40% of stage (negligible)	good
Loop closing					
Place recognition	outcome	candidates accepted	4 of 281	<i>convention:</i> >10 % of candidates accepted (2494 events)	poor
Loop fusion + essential graph	ablation	ATE(on)/ATE(off)	1.37	<i>convention:</i> <1.0 (enabling it should help) (one run per configuration)	poor
Full BA	outcome	global BA executions	–	– (see table 4.3)	no claim

Table 4.22: Per-block evidence for **VINS-Fusion** in the static condition, figure 4.13. Five **allsensor** datasets, median across runs.

Block	Rule	Metric	Value	Criterion / note	Verdict
Front end					
Feature tracking	latency	median feature_tracking_ms	55.2	<66.0 ms frame budget	good
Feature queue	latency	median feature_queue_wait_ms	403.7	<66.0 ms frame budget (backlog 4.0 frames)	poor
Persistent-feature removal (YOLO)	none	–	–	– (filter disabled in every run reported here)	no claim
Back end					
IMU propagation	latency	median imu_propagation_ms	0.272	<i>convention:</i> <40% of stage (0.4% of stage)	good
Triangulation	latency	median triangulation_ms	0.063	<i>convention:</i> <40% of stage (0.1% of stage)	good
Problem construction	latency	median problem_construction_ms	3.2	<i>convention:</i> <40% of stage (4.7% of stage)	good
Ceres solve	latency	median solver_ms	48.3	<i>convention:</i> <40% of stage (71.9% of stage)	poor
Marginalization	latency	median marginalization_ms	4.1	<i>convention:</i> <40% of stage (6.1% of stage)	good
Outlier rejection	latency	median outlier_rejection_ms	0.155	<i>convention:</i> <40% of stage (0.2% of stage)	good
Slide window	latency	median slide_window_ms	0.212	<i>convention:</i> <40% of stage (0.3% of stage)	good
BACK END total	latency	median backend_total_ms	67.2	<66.0 ms frame budget	poor
Solver convergence	outcome	solves returning a usable solution	99.64%	<i>convention:</i> ≥99 % usable (22.8 % hit the iteration or 80 ms cap without converging; reported, not graded)	good
Loop closing					
Loop fusion	none	–	–	– (not implemented in this fork)	no claim
RTAB-Map correction	none	–	–	– (separate process; correction never injected)	no claim

Table 4.23: Per-block evidence for the **EKF wheel + IMU** baseline, [figure 4.13](#). Every row but the last two is an ablation: the block was removed or swapped and all five datasets re-run, so these are the only verdicts in this section that speak to trajectory error.

Block	Rule	Metric	Value	Criterion / note	Verdict
EKF					
Wheel speed correction	ablation	ATE without the wheel update / with	54.08	<i>convention:</i> $\geq 2.0\times$ (block is load-bearing) (median over 5 bags; joint with the other row – one flag removes both)	good
Gyro-bias observation	ablation	ATE without the wheel update / with	54.08	<i>convention:</i> $\geq 2.0\times$ (block is load-bearing) (median over 5 bags; joint with the other row – one flag removes both)	good
IMU prediction	ablation	ATE of wheel-only / EKF	6.56	<i>convention:</i> $\geq 2.0\times$ (block is load-bearing) (median over 5 bags)	good
Yaw-rate model (bicycle)	ablation	ATE with differential / with bicycle	1.02	<i>convention:</i> $\geq 2.0\times$ (block is load-bearing) (median over 5 bags)	poor
Initial pose anchor	none	–	–	– (ground-truth pose; not a performance block)	no claim
Covariance propagation	outcome	ANEES (3 DOF)	–	<i>derived:</i> 3.0 for a consistent filter (64.7–123.2; see table 4.7)	poor



The EKF wheel + IMU baseline has no row here: no dynamic dataset carries wheel encoders or wheel odometry, so it cannot be run in this condition at all (table 3.9).



Figure 4.14: Per-block performance in the **dynamic** condition, over the twelve relogged dynamic runs of each estimator. Loop closing carries no ablation here because the relogged campaign has no loop-closure-disabled variant. Evidence in tables 4.24 and 4.25, one table per algorithm.

Table 4.24: Per-block evidence for **ORB-SLAM3** in the dynamic condition, [figure 4.14](#). Twelve relogged dynamic runs, median across runs.

Block	Rule	Metric	Value	Criterion / note	Verdict
Tracking					
Extract ORB (+preproc, stereo)	latency	median image_preprocessing_feature_stereo_ms	45.1	<i>convention:</i> <40% of stage (73.8% of stage)	poor
IMU preintegration	latency	median imu_preintegration_ms	0.116	<i>convention:</i> <40% of stage (0.2% of stage)	good
Initial pose estimation	latency	median pose_prediction_ms	0.055	<i>convention:</i> <40% of stage (0.1% of stage)	good
Track local map	latency	median local_map_tracking_ms	12.2	<i>convention:</i> <40% of stage (19.9% of stage)	good
New keyframe decision	latency	median keyframe_decision_ms	0.179	<i>convention:</i> <40% of stage (0.3% of stage)	good
TRACKING total	latency	median tracking_total_ms	61.2	<i>derived:</i> <66.0 ms frame budget	good
Dynamic feature removal (YOLO)	none	–	–	– (filter disabled in every run reported here)	no claim
Local mapping					
KeyFrame insertion	latency	median keyframe_insertion_ms	10.9	<i>convention:</i> <40% of stage (5.7% of stage)	good
Recent MapPoints culling	latency	median map_point_culling_ms	0.714	<i>convention:</i> <40% of stage (0.4% of stage)	good
New points creation	latency	median map_point_creation_fusion_ms	50.0	<i>convention:</i> <40% of stage (26.3% of stage)	good
Local BA	latency	median local_ba_ms	122.9	<i>convention:</i> <40% of stage (64.6% of stage)	poor
Local keyframes culling	latency	median keyframe_culling_ms	4.0	<i>convention:</i> <40% of stage (2.1% of stage)	good
IMU initialization	latency	median imu_initialization_ms	–	<i>convention:</i> <40% of stage (not logged)	no claim
IMU scale refinement	latency	median inertial_refinement_ms	–	<i>convention:</i> <40% of stage (not logged)	no claim
Loop closing					
Place recognition	outcome	candidates accepted	0 of 2	<i>convention:</i> >10 % of candidates accepted (5726 events)	poor
Loop fusion + essential graph	ablation	ATE(on)/ATE(off)	–	<i>convention:</i> <1.0 (enabling it should help) (one run per configuration)	no claim
Full BA	outcome	global BA executions	–	– (see table 4.3)	no claim

Table 4.25: Per-block evidence for **VINS-Fusion** in the dynamic condition, [figure 4.14](#). Twelve relogged dynamic runs, median across runs.

Block	Rule	Metric	Value	Criterion / note	Verdict
Front end					
Feature tracking	latency	median feature_tracking_ms	42.5	<66.0 ms frame budget	good
Feature queue	latency	median feature_queue_wait_ms	502.1	<66.0 ms frame budget (backlog 7.5 frames)	poor
Persistent-feature removal (YOLO)	none	–	–	– (filter disabled in every run reported here)	no claim
Back end					
IMU propagation	latency	median imu_propagation_ms	0.204	<i>convention:</i> <40% of stage (0.3% of stage)	good
Triangulation	latency	median triangulation_ms	0.062	<i>convention:</i> <40% of stage (0.1% of stage)	good
Problem construction	latency	median problem_construction_ms	1.1	<i>convention:</i> <40% of stage (1.7% of stage)	good
Ceres solve	latency	median solver_ms	57.2	<i>convention:</i> <40% of stage (89.8% of stage)	poor
Marginalization	latency	median marginalization_ms	2.4	<i>convention:</i> <40% of stage (3.7% of stage)	good
Outlier rejection	latency	median outlier_rejection_ms	0.068	<i>convention:</i> <40% of stage (0.1% of stage)	good
Slide window	latency	median slide_window_ms	0.167	<i>convention:</i> <40% of stage (0.3% of stage)	good
BACK END total	latency	median backend_total_ms	63.7	<66.0 ms frame budget	good
Solver convergence	outcome	solves returning a usable solution	99.55%	<i>convention:</i> ≥99% usable (45.1% hit the iteration or 80 ms cap without converging; reported, not graded)	good
Loop closing					
Loop fusion	none	–	–	– (not implemented in this fork)	no claim
RTAB-Map correction	none	–	–	– (separate process; correction never injected)	no claim

Four readings follow that the trajectory tables alone do not give.

Both visual front ends are the dominant cost, and neither keeps up cleanly. ORB-SLAM3 spends 33.1 ms of a 53.8 ms tracking budget on preprocessing, extraction and stereo matching in static scenes, rising to 45.1 ms of 61.2 ms in dynamic ones. VINS-Fusion’s back end waits 403.7 ms per frame on its feature queue in static scenes and 502.1 ms in dynamic ones, against a 66.0 ms frame budget, with a backlog of four to seven frames. **These are the same deficit wearing different clothes:** neither estimator processes frames as fast as they arrive, and the two differ in what they do about it. ORB-SLAM3 subscribes best-effort at depth five and discards the overflow, which is the frame shortfall of [section 4.1](#); VINS-Fusion subscribes reliably at depth 100 and absorbs it as latency. The frame counters see one and not the other.

Local BA dominates ORB-SLAM3’s local mapping at 244.4 ms of 312.9 ms in static scenes. It runs per keyframe rather than per frame, so it does not compete directly with the frame budget, but it is where that pipeline’s compute goes.

Loop closing is not earning its place in these scenes. Across the five `allsensor` datasets, enabling it leaves the median aligned ATE 1.37 times *worse* than disabling it, and only 4 of 281 place-recognition candidates were ever accepted ([table 4.3](#)). This is the project’s one ORB-SLAM3 ablation, and it is still one run per configuration, so it sits inside an unmeasured run-to-run spread—the verdict is reported, not relied upon.

The EKF’s accuracy rests on one intervention. Removing the wheel update costs a factor of 54 in aligned ATE; removing the IMU prediction costs 6.6. The 54 covers the wheel speed correction and the gyroscope-bias observation *together*: both are measurement rows of the same update, one flag removes both, and no flag removes only one, so [table 4.23](#) lists the figure against each row while marking it as joint rather than presenting two independent results. Swapping the yaw-rate model from the bicycle formulation to the rear-wheel differential, by contrast, changes nothing measurable—median 1.0, and on `allsensor_004` the noisier estimator is the better one. That is worth stating plainly, because the differential model is four times worse as a *rate* estimator ([table 3.10](#)) and dominates standalone wheel odometry’s error. Inside the filter it barely matters, because the wheel rate is used only to observe the gyroscope bias and is weighted by its own measured σ . The design choice of [section 3.6](#) is vindicated by its own ablation.

The campaign succeeds as an instrumentation milestone: repeated run-level data, front-end and back-end timing, input retention, process resources, initialization, tracking loss, keyframes, loop closures, map merges, and raw trajectories are now available in auditable form. Ground-truth accuracy can now be measured, but the campaign does not support the intended semantic-filter hypothesis or a clean dynamic-scene estimator ranking. The limitations are:

[Table 4.26](#) lists them with the consequence each one carries.

No root cause is assigned in this report. In particular, differences are not attributed to calibration, timestamp handling, masks, replay scheduling, or a specific estimator component without a controlled diagnostic experiment. The immediate advisor decision is whether the next work period should first restore active, input-matched filtering and plausible full trajectories, or instead prioritize the planned integration of relocalization, global optimization, and map merging. The latter remains future work and would provide a unified localization/map input for navigation; path planning itself remains outside scope.

Table 4.26: Limitations of the results reported in this chapter, and what each one prevents. “Scope” identifies the experiment affected: R is the relogged campaign, E the estimator comparison of [section 4.2](#), and A both.

Scope	Limitation	Consequence
A	RPE was missing	now implemented and reported (section 4.5.1); it withdrew this chapter’s local-coherence reading
R	every dynamic trajectory fails the jump-plausibility criterion	no dynamic estimator ranking
R	the VINS semantic treatment was inactive in all repeats	no VINS filtering conclusion
R	11 of 12 ORB treatment pairs fail the paired-input rule	no cross-treatment accuracy conclusion
R	YOLO truth is projected simulator geometry, not annotated visible extent, and the retained confidence threshold is fixed	no AP claim
R	class metrics are conditional on successful IoU localization	must not be read as end-to-end accuracy
R	two static ORB run summaries have incomplete schemas	retained as operational; counters unavailable
A	CPU and memory are per-estimator-process figures	not whole-system cost
A	one laptop and one replay rate throughout	no hardware or input-rate stress surface
E	replay rate 0.5× for both visual estimators	accuracy figures are not evidence of real-time capability
E	baselines are offline, visual estimators ran live	timing-induced error is present on one side of the comparison only
E	only allsensor bags carry wheel encoders	the baseline exists for static featureless-floor scenes only; no dynamic condition
E	$n = 5$ across four worlds, no condition repeated more than twice	no mean, deviation or interval; between-bag differences are not effects
E	ORB-SLAM3 drops frames at a depth-5 best-effort subscriber; VINS buffers 100 deep	input delivery is not matched between the two; no timing comparison is admissible
E	ORB-SLAM3 run-to-run spread unmeasured, one run per configuration	loop-closure differences cannot be attributed; repeats required
E	loop-closure state absent from run_metadata.csv	configurations distinguishable only by inspecting loop_closing.csv
E	origin of the wheel yaw-rate offset untested	the drift is predicted but not yet attributable or correctable (section 4.2.4)
E	wheel-rate offset is absorbed by the EKF gyroscope-bias state	now quantified: EKF heading error rises tenfold where the wheel rate is poor (table 4.6)
E	accelerometer characterisation unstable across bags	bias 0.019 to 0.068, scale 1.020 to 1.067 for one simulated sensor
E	neither visual estimator writes a pose covariance	the consistency test cannot be applied to them

Analysis Artifacts

The numerical sources for this chapter are `output/compare/relogged_20260916/run_metrics.csv`, `module_latency_summary.csv`, `trajectory_metrics.csv`, and `pair_validity.csv`. The anomaly evidence is preserved in `anomaly_catalog.csv` and `anomaly_run_metrics.csv`. The reproducible offline renderers are `script/analyse_relogged_runs.py` and `script/analyse_relogged_anomalies.py`. The per-run and example-run tables in this chapter are typeset from `trajectory_metrics.csv` by `script/make_result_tables.py`, which writes `report/minor_report/tables/`; they are not hand-entered numbers. Dense-map evidence is `map/map_tiny.db` with `output/compare/rtab_map_vs_gt.png` (independent run) and `rtab_map_vs_gt_selfcheck.png` (build run), produced by `script/validate_map.py` with the placement measured by `script/fit_map_pose.py`; the graph counts and the node-pose agreement were read from the database read-only. Relative pose error is in `rpe_metrics.csv` (interval sweep) with the 1s values carried into `trajectory_metrics.csv`; both tables are rendered by `script/make_result_tables.py`. The evaluator prints its metrics to the console and writes no machine-readable summary of its own, so both runs are transcribed to `output/compare/rtab_map_vs_gt_metrics.csv`, with the second run's console output preserved verbatim in `rtab_map_vs_gt_selfcheck.txt`. The first run's console output was not captured and its ground-truth bag no longer exists, so its row is transcribed from the engineering record. Offline detector evidence is stored below `output/yolo_eval/dyn_00_000/`, `dyn_00_001/`, `dyn_01_000/`, and `dyn_02_000/`; the pooled source is `summary_metrics.csv`, and the pooled detection association counts are in `confusion_matrix.csv`. Conditional object-class results are in `classification_confusion_matrix.csv` and `classification_metrics.csv`. Its evaluator and summarizer are `script/yolo_eval.py` and `script/summarise_yolo_eval.py`. Light and dark figures, including one comparison figure for every dataset/repeat group, are stored below `output/compare/relogged_20260916/`; its `ground_truth/` subdirectory contains the seven extracted rosbag truth trajectories. Raw estimator directories and raw bags were not modified.